13 Neural Networks for Tabular Data

13.1 Introduction

In Part II, we discussed linear models for regression and classification. In particular, in Chapter 10, we discussed logistic regression, which, in the binary case, corresponds to the model $p(y|\boldsymbol{x}, \boldsymbol{w}) = \text{Ber}(y|\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}))$, and in the multiclass case corresponds to the model $p(y|\boldsymbol{x}, \mathbf{W}) = \text{Cat}(y|\text{softmax}(\mathbf{W}\boldsymbol{x}))$. In Chapter 11, we discussed linear regression, which corresponds to the model $p(y|\boldsymbol{x}, \boldsymbol{w}) = \mathcal{N}(y|\boldsymbol{w}^\mathsf{T}\boldsymbol{x}, \sigma^2)$. And in Chapter 12, we discussed generalized linear models, which generalizes these models to other kinds of output distributions, such as Poisson. However, all these models make the strong assumption that the input-output mapping is linear.

A simple way of increasing the flexibility of such models is to perform a feature transformation, by replacing $\boldsymbol{x}$ with $\boldsymbol{\phi}(\boldsymbol{x})$. For example, we can use a polynomial transform, which in 1d is given by $\boldsymbol{\phi}(x) = [1, x, x^2, x^3, \ldots]$, as we discussed in Section 1.2.2.2. This is sometimes called **basis function expansion**. The model now becomes

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}\boldsymbol{\phi}(\boldsymbol{x}) + \boldsymbol{b} \tag{13.1}$$

This is still linear in the parameters $\boldsymbol{\theta} = (\mathbf{W}, \boldsymbol{b})$, which makes model fitting easy (since the negative log-likelihood is convex). However, having to specify the feature transformation by hand is very limiting.

A natural extension is to endow the feature extractor with its own parameters, $\boldsymbol{\theta}_2$, to get

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}\boldsymbol{\phi}(\boldsymbol{x}; \boldsymbol{\theta}_2) + \boldsymbol{b} \tag{13.2}$$

where $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$ and $\boldsymbol{\theta}_1 = (\mathbf{W}, \boldsymbol{b})$. We can obviously repeat this process recursively, to create more and more complex functions. If we compose L functions, we get

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = f_L(f_{L-1}(\cdots(f_1(\boldsymbol{x}))\cdots)) \tag{13.3}$$

where $f_\ell(\boldsymbol{x}) = f(\boldsymbol{x}; \boldsymbol{\theta}_\ell)$ is the function at layer ℓ. This is the key idea behind **deep neural networks** or **DNN**s.

The term "DNN" actually encompasses a larger family of models, in which we compose differentiable functions into any kind of DAG (directed acyclic graph), mapping input to output. Equation (13.3) is the simplest example where the DAG is a chain. This is known as a **feedforward neural network** (**FFNN**) or **multilayer perceptron** (**MLP**).

An MLP assumes that the input is a fixed-dimensional vector, say $\boldsymbol{x} \in \mathbb{R}^D$. It is common to call such data "**structured data**" or "**tabular data**", since the data is often stored in an $N \times D$ design

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Table 13.1: Truth table for the XOR (exclusive OR) function, $y = x_1 \veebar x_2$.

matrix, where each column (feature) has a specific meaning, such as height, weight, age, etc. In later chapters, we discuss other kinds of DNNs that are more suited to "**unstructured data**" such as images and text, where the input data is variable sized, and each individual element (e.g., pixel or word) is often meaningless on its own.[1] In particular, in Chapter 14, we discuss **convolutional neural networks** (**CNN**), which are designed to work with images; in Chapter 15, we discuss **recurrent neural networks** (**RNN**) and **transformers**, which are designed to work with sequences; and in Chapter 23, we discuss **graph neural networks** (**GNN**), which are designed to work with graphs.

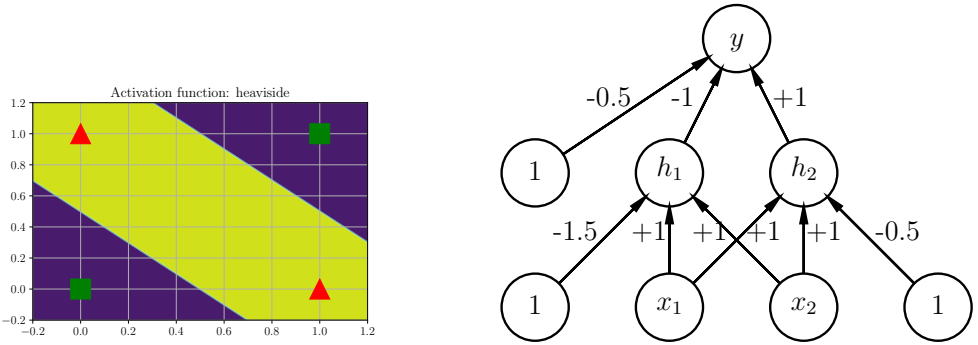
Although DNNs can work well, there are often a lot of engineering details that need to be addressed to get good performance. Some of these details are discussed in the supplementary material to this book, available at probml.ai. There are also various other books that cover this topic in more depth (e.g., [Zha+20; Cho21; Gér19; GBC16; Raf22]), as well as a multitude of online courses. For a more theoretical treatment, see e.g., [Ber+21; Cal20; Aro+21; RY21].

13.2 Multilayer perceptrons (MLPs)

In Section 10.2.5, we explained that a **perceptron** is a deterministic version of logistic regression. Specifically, it is a mapping of the following form:

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbb{I}\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b \geq 0\right) = H(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b) \tag{13.4}$$

where $H(a)$ is the **heaviside step function**, also known as a **linear threshold function**. Since the decision boundaries represented by perceptrons are linear, they are very limited in what they can represent. In 1969, Marvin Minsky and Seymour Papert published a famous book called *Perceptrons* [MP69] in which they gave numerous examples of pattern recognition problems which perceptrons cannot solve. We give a specific example below, before discussing how to solve the problem.



(a) (b)

Figure 13.1: (a) Illustration of the fact that the XOR function is not linearly separable, but can be separated by the two layer model using Heaviside activation functions. Adapted from Figure 10.6 of [Gér19]. Generated by xor_heaviside.ipynb. (b) A neural net with one hidden layer, whose weights have been manually constructed to implement the XOR function. h_1 is the AND function and h_2 is the OR function. The bias terms are implemented using weights from constant nodes with the value 1.

13.2.1 The XOR problem

One of the most famous examples from the *Perceptrons* book is the **XOR problem**. Here the goal is to learn a function that computes the exclusive OR of its two binary inputs. The truth table for this function is given in Table 13.1. We visualize this function in Figure 13.1a. It is clear that the data is not linearly separable, so a perceptron cannot represent this mapping.

However, we can overcome this problem by stacking multiple perceptrons on top of each other. This is called a **multilayer perceptron** (**MLP**). For example, to solve the XOR problem, we can use the MLP shown in Figure 13.1b. This consists of 3 perceptrons, denoted h_1, h_2 and y. The nodes marked x are inputs, and the nodes marked 1 are constant terms. The nodes h_1 and h_2 are called **hidden units**, since their values are not observed in the training data.

The first hidden unit computes $h_1 = x_1 \wedge x_2$ by using appropriately set weights. (Here $\wedge$ is the AND operation.) In particular, it has inputs from x_1 and x_2, both weighted by 1.0, but has a bias term of -1.5 (this is implemented by a "wire" with weight -1.5 coming from a dummy node whose value is fixed to 1). Thus h_1 will fire iff x_1 and x_2 are both on, since then

$$\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x} - b_1 = [1.0, 1.0]^\mathsf{T}[1, 1] - 1.5 = 0.5 > 0 \tag{13.5}$$

1. The term "unstructured data" is a bit misleading, since images and text *do* have structure. For example, neighboring pixels in an image are highly correlated, as are neighboring words in a sentence. Indeed, it is precisely this structure that is exploited (assumed) by CNNs and RNNs. By contrast, MLPs make no assumptions about their inputs. This is useful for applications such as tabular data, where the structure (dependencies between the columns) is usually not obvious, and thus needs to be learned. We can also apply MLPs to images and text, as we will see, but performance will usually be worse compared to specialized models, such as as CNNs and RNNs. (There are some exceptions, such as the **MLP-mixer** model of [Tol+21], which is an unstructured model that can learn to perform well on image and text data, but such models need massive datasets to overcome their lack of inductive bias.)

Similarly, the second hidden unit computes $h_2 = x_1 \vee x_2$, where $\vee$ is the OR operation, and the third computes the output $y = \overline{h_1} \wedge h_2$, where $\overline{h} = \neg h$ is the NOT (logical negation) operation. Thus y computes

$$y = f(x_1, x_2) = \overline{(x_1 \wedge x_2)} \wedge (x_1 \vee x_2) \tag{13.6}$$

This is equivalent to the XOR function.

By generalizing this example, we can show that an MLP can represent any logical function. However, we obviously want to avoid having to specify the weights and biases by hand. In the rest of this chapter, we discuss ways to learn these parameters from data.

13.2.2 Differentiable MLPs

The MLP we discussed in Section 13.2.1 was defined as a stack of perceptrons, each of which involved the non-differentiable Heaviside function. This makes such models difficult to train, which is why they were never widely used. However, suppose we replace the Heaviside function $H : \mathbb{R} \to \{0, 1\}$ with a differentiable **activation function** $\varphi : \mathbb{R} \to \mathbb{R}$. More precisely, we define the hidden units $\boldsymbol{z}_l$ at each layer l to be a linear transformation of the hidden units at the previous layer passed elementwise through this activation function:

$$\boldsymbol{z}_l = f_l(\boldsymbol{z}_{l-1}) = \varphi_l\left(\boldsymbol{b}_l + \mathbf{W}_l \boldsymbol{z}_{l-1}\right) \tag{13.7}$$

or, in scalar form,

$$z_{kl} = \varphi_l\left(b_{kl} + \sum_{j=1}^{K_{l-1}} w_{lkj} z_{jl-1}\right) \tag{13.8}$$

The quantity that is passed to the activation function is called the **pre-activations**:

$$\boldsymbol{a}_l = \boldsymbol{b}_l + \mathbf{W}_l \boldsymbol{z}_{l-1} \tag{13.9}$$

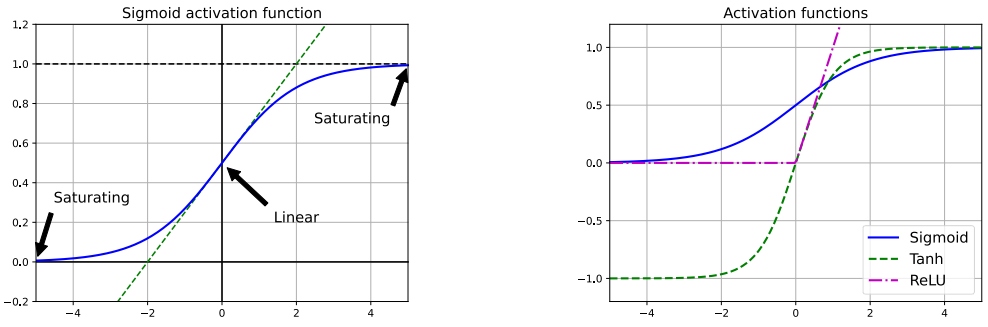
so $\boldsymbol{z}_l = \varphi_l(\boldsymbol{a}_l)$.

If we now compose L of these functions together, as in Equation (13.3), then we can compute the gradient of the output wrt the parameters in each layer using the chain rule, also known as **backpropagation**, as we explain in Section 13.3. (This is true for any kind of differentiable activation function, although some kinds work better than others, as we discuss in Section 13.2.3.) We can then pass the gradient to an optimizer, and thus minimize some training objective, as we discuss in Section 13.4. For this reason, the term "MLP" almost always refers to this differentiable form of the model, rather than the historical version with non-differentiable linear threshold units.

13.2.3 Activation functions

We are free to use any kind of differentiable activation function we like at each layer. However, if we use a *linear* activation function, $\varphi_\ell(a) = c_\ell a$, then the whole model reduces to a regular linear model. To see this, note that Equation (13.3) becomes

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}_L c_L(\mathbf{W}_{L-1} c_{L-1}(\cdots(\mathbf{W}_1 \boldsymbol{x})\cdots)) \propto \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \boldsymbol{x} = \mathbf{W}' \boldsymbol{x} \tag{13.10}$$



(a) (b)

Figure 13.2: (a) Illustration of how the sigmoid function is linear for inputs near 0, but saturates for large positive and negative inputs. Adapted from 11.1 of [Gér19]. (b) Plots of some neural network activation functions. Generated by activation_fun_plot.ipynb.

where we dropped the bias terms for notational simplicity. For this reason, it is important to use nonlinear activation functions.

In the early days of neural networks, a common choice was to use a sigmoid (logistic) function, which can be seen as a smooth approximation to the Heaviside function used in a perceptron:

$$\sigma(a) = \frac{1}{1 + e^{-a}} \tag{13.11}$$

However, as shown in Figure 13.2a, the sigmoid function **saturates** at 1 for large positive inputs, and at 0 for large negative inputs. Another common choice is the tanh function, which has a similar shape, but saturates at -1 and +1. See Figure 13.2b.

In the saturated regimes, the gradient of the output wrt the input will be close to zero, so any gradient signal from higher layers will not be able to propagate back to earlier layers. This is called the **vanishing gradient problem**, and it makes it hard to train the model using gradient descent (see Section 13.4.2 for details). One of the keys to being able to train very deep models is to use non-saturating activation functions. Several different functions have been proposed. The most common is **rectified linear unit** or **ReLU**, proposed in [GBB11; KSH12]. This is defined as

$$\text{ReLU}(a) = \max(a, 0) = a\mathbb{I}(a > 0) \tag{13.12}$$

The ReLU function simply "turns off" negative inputs, and passes positive inputs unchanged: see Figure 13.2b for a plot, and Section 13.4.3 for more details.

When neural networks are used to represent functions defined on a continuous input space — such as points in time, $f(t)$, or in 3d space, $f(x, y, z)$ — they are often called **neural implicit representations** or **coordinated based representations** of the underlying signal. In such cases, it is often important to capture high frequencies to represent the signal faithfully. Unfortunately MLPs have an intrinsic bias to low frequency functions [Tan+20; RML22]. One simple solution is to use a sine function, $\sin(a)$, as the nonlinearity, instead of ReLU, as explained in [Sit+20].[2]

2. For some simple illustrations of the surprising power of the sine activation function for learning functions

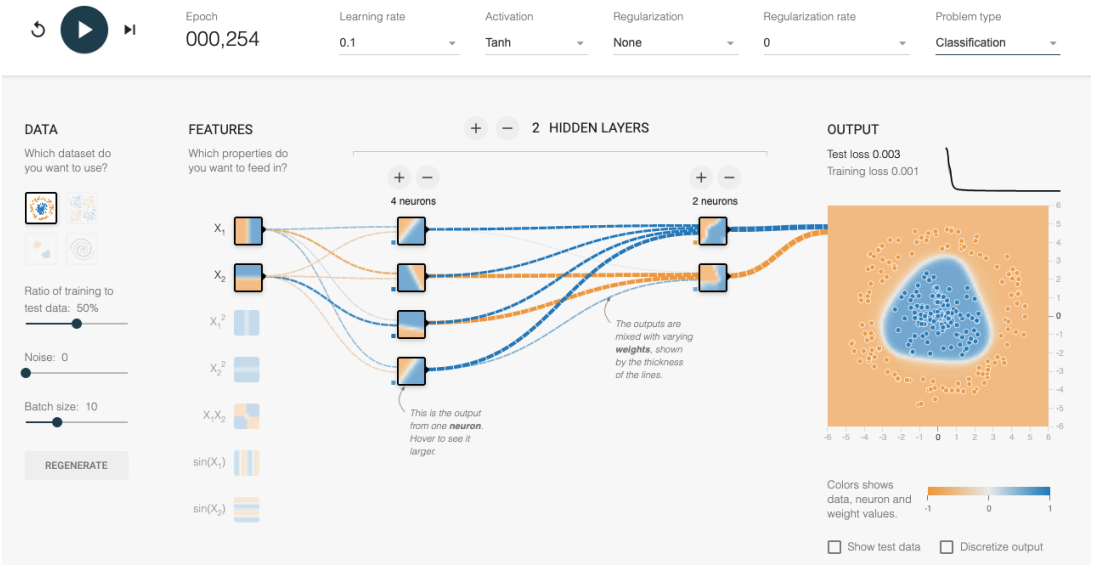


Figure 13.3: An MLP with 2 hidden layers applied to a set of 2d points from 2 classes, shown in the top left corner. The visualizations associated with each hidden unit show the decision boundary at that part of the network. The final output is shown on the right. The input is $\boldsymbol{x} \in \mathbb{R}^2$, the first layer activations are $\boldsymbol{z}_1 \in \mathbb{R}^4$, the second layer activations are $\boldsymbol{z}_2 \in \mathbb{R}^2$, and the final logit is $a_3 \in \mathbb{R}$, which is converted to a probability using the sigmoid function. This is a screenshot from the interactive demo at http://playground.tensorflow.org.

13.2.4 Example models

MLPs can be used to perform classification and regression for many kinds of data. We give some examples below.

13.2.4.1 MLP for classifying 2d data into 2 categories

Figure 13.3 gives an illustration of an MLP with two hidden layers applied to a 2d input vector, corresponding to points in the plane, coming from two concentric circles. This model has the following form:

$$p(y|\boldsymbol{x};\boldsymbol{\theta}) = \text{Ber}(y|\sigma(a_3)) \tag{13.13}$$

$$a_3 = \boldsymbol{w}_3^\intercal \boldsymbol{z}_2 + b_3 \tag{13.14}$$

$$\boldsymbol{z}_2 = \varphi(\mathbf{W}_2 \boldsymbol{z}_1 + \boldsymbol{b}_2) \tag{13.15}$$

$$\boldsymbol{z}_1 = \varphi(\mathbf{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) \tag{13.16}$$

Here a_3 is the final logit score, which is converted to a probability via the sigmoid (logistic) function. The value a_3 is computed by taking a linear combination of the 2 hidden units in layer 2, using

on low dimensional input spaces, see https://nipunbatra.github.io/blog/posts/siren-paper.html and https://nipunbatra.github.io/blog/posts/siren-paper.html.

```
  Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 128)               100480
_________________________________________________________________
dense_1 (Dense)              (None, 128)               16512
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0
```

Table 13.2: Structure of the MLP used for MNIST classification. Note that $100,480 = (784+1) \times 128$*, and* $16,512 = (128+1) \times 128$*. mlp_mnist_tf.ipynb.*

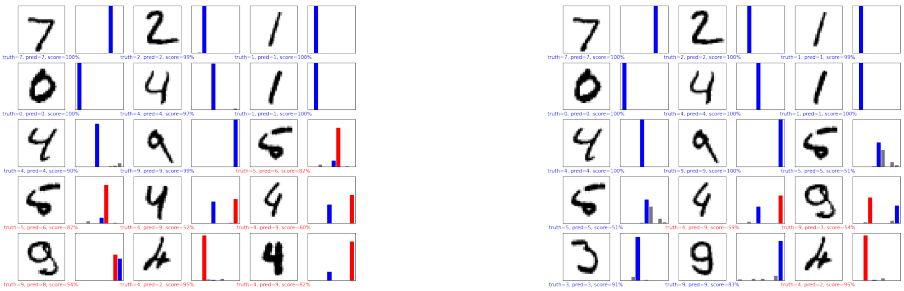
$a_3 = \boldsymbol{w}_3^\mathsf{T}\boldsymbol{z}_2 + b_3$. In turn, layer 2 is computed by taking a nonlinear combination of the 4 hidden units in layer 1, using $\boldsymbol{z}_2 = \varphi(\mathbf{W}_2\boldsymbol{z}_1 + \boldsymbol{b}_2)$. Finally, layer 1 is computed by taking a nonlinear combination of the 2 input units, using $\boldsymbol{z}_1 = \varphi(\mathbf{W}_1\boldsymbol{x} + \boldsymbol{b}_1)$. By adjusting the parameters, $\boldsymbol{\theta} = (\mathbf{W}_1, \boldsymbol{b}_1, \mathbf{W}_2, \boldsymbol{b}_2, \boldsymbol{w}_3, b_3)$, to minimize the negative log likelihood, we can fit the training data very well, despite the highly nonlinear nature of the decision boundary. (You can find an interactive version of this figure at http://playground.tensorflow.org.)

13.2.4.2 MLP for image classification

To apply an MLP to image classification, we need to "**flatten**" the 2d input into 1d vector. We can then use a feedforward architecture similar to the one described in Section 13.2.4.1. For example, consider building an MLP to classifiy MNIST digits (Section 3.5.2). These are $28 \times 28 = 784$-dimensional. If we use 2 hidden layers with 128 units each, followed by a final 10 way softmax layer, we get the model shown in Table 13.2.

We show some predictions from this model in Figure 13.4. We train it for just two "epochs" (passes over the dataset), but already the model is doing quite well, with a test set accuracy of 97.1%. Furthermore, the errors seem sensible, e.g., 9 is mistaken as a 3. Training for more epochs can further improve test accuracy.

In Chapter 14 we discuss a different kind of model, called a convolutional neural network, which is better suited to images. This gets even better performance and uses fewer parameters, by exploiting prior knowledge about the spatial structure of images. By contrast, with an MLP, we can randomly shuffle (permute) the pixels without affecting the output (assuming we use the same random permutation for all inputs).



(a) (b)

Figure 13.4: Results of applying an MLP (with 2 hidden layers with 128 units and 1 output layer with 10 units) to some MNIST images (cherry picked to include some errors). Red is incorrect, blue is correct. (a) After 1 epoch of training. (b) After 2 epochs. Generated by mlp_mnist_tf.ipynb.

13.2.4.3 MLP for text classification

To apply MLPs to text classification, we need to convert the variable-length sequence of words $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_T$ (where each $\boldsymbol{v}_t$ is a one-hot vector of length V, where V is the vocabulary size) into a fixed dimensional vector $\boldsymbol{x}$. The easiest way to do this is as follows. First we treat the input as an unordered bag of words (Section 1.5.4.1), $\{\boldsymbol{v}_t\}$. The first layer of the model is a $E \times V$ embedding matrix $\mathbf{W}_1$, which converts each sparse V-dimensional vector to a dense E-dimensional embedding, $\boldsymbol{e}_t = \mathbf{W}_1 \boldsymbol{v}_t$ (see Section 20.5 for more details on word embeddings). Next we convert this set of T E-dimensional embeddings into a fixed-sized vector using **global average pooling**, $\overline{\boldsymbol{e}} = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{e}_t$. This can then be passed as input to an MLP. For example, if we use a single hidden layer, and a logistic output (for binary classification), we get

$$p(y|\boldsymbol{x};\boldsymbol{\theta}) = \text{Ber}(y|\sigma(\boldsymbol{w}_3^\intercal \boldsymbol{h} + b_3)) \tag{13.17}$$

$$\boldsymbol{h} = \varphi(\mathbf{W}_2 \overline{\boldsymbol{e}} + \boldsymbol{b}_2) \tag{13.18}$$

$$\overline{\boldsymbol{e}} = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{e}_t \tag{13.19}$$

$$\boldsymbol{e}_t = \mathbf{W}_1 \boldsymbol{v}_t \tag{13.20}$$

If we use a vocabulary size of $V = 10,000$, an embedding size of $E = 16$, and a hidden layer of size 16, we get the model shown in Table 13.3. If we apply this to the IMDB movie review sentiment classification dataset discussed in Section 1.5.2.1, we get 86% on the validation set.

We see from Table 13.3 that the model has a lot of parameters, which can result in overfitting, since the IMDB training set only has 25k examples. However, we also see that most of the parameters are in the embedding matrix, so instead of learning these in a supervised way, we can perform unsupervised pre-training of word embedding models, as we discuss in Section 20.5. If the embedding matrix $\mathbf{W}_1$ is fixed, we just have to fine-tune the parameters in layers 2 and 3 for this specific labeled task, which requires much less data. (See also Chapter 19, where we discuss general techniques for training with limited labeled data.)

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, None, 16)          160000
_________________________________________________________________
global_average_pooling1d (Gl (None, 16)                0
_________________________________________________________________
dense (Dense)                (None, 16)                272
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 17
=================================================================
Total params: 160,289
Trainable params: 160,289
Non-trainable params: 0
```

Table 13.3: Structure of the MLP used for IMDB review classification. We use a vocabulary size of $V = 10,000$, an embedding size of $E = 16$, and a hidden layer of size 16. The embedding matrix $\mathbf{W}_1$ has size $10,000 \times 16$, the hidden layer (labeled "dense") has a weight matrix $\mathbf{W}_2$ of size 16×16 and bias $\boldsymbol{b}_2$ of size 16 (note that $16 \times 16 + 16 = 272$), and the final layer (labeled "dense_1") has a weight vector $\boldsymbol{w}_3$ of size 16 and a bias b_3 of size 1. The global average pooling layer has no free parameters. mlp_imdb_tf.ipynb.

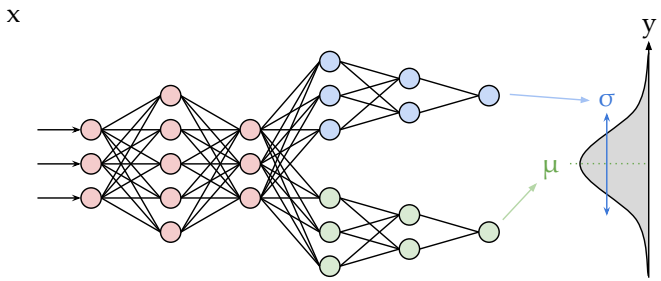
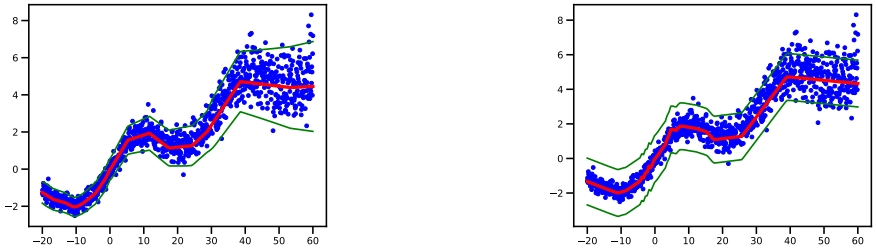


Figure 13.5: Illustration of an MLP with a shared "backbone" and two output "heads", one for predicting the mean and one for predicting the variance. From https://brendanhasz.github.io/2019/07/23/bayesian-density-net.html. Used with kind permission of Brendan Hasz.

13.2.4.4 MLP for heteroskedastic regression

We can also use MLPs for regression. Figure 13.5 shows how we can make a model for heteroskedastic nonlinear regression. (The term "heteroskedastic" just means that the predicted output variance is input-dependent, as discussed in Section 2.6.3.) This function has two outputs which compute $f_\mu(\boldsymbol{x}) = \mathbb{E}[y|\boldsymbol{x}, \boldsymbol{\theta}]$ and $f_\sigma(\boldsymbol{x}) = \sqrt{\mathbb{V}[y|\boldsymbol{x}, \boldsymbol{\theta}]}$. We can share most of the layers (and hence parameters) between these two functions by using a common "**backbone**" and two output "**heads**", as shown in Figure 13.5. For the μ head, we use a linear activation, $\varphi(a) = a$. For the σ head, we use a softplus



(a) (b)

Figure 13.6: Illustration of predictions from an MLP fit using MLE to a 1d regression dataset with growing noise. (a) Output variance is input-dependent, as in Figure 13.5. (b) Mean is computed using same model as in (a), but output variance is treated as a fixed parameter σ^2, which is estimated by MLE after training, as in Section 11.2.3.6. Generated by mlp_1d_regression_hetero_tfp.ipynb.

activation, $\varphi(a) = \sigma_+(a) = \log(1 + e^a)$. If we use linear heads and a nonlinear backbone, the overall model is given by

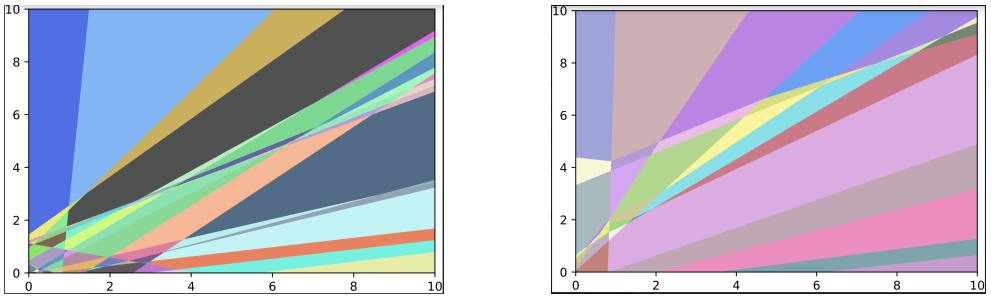
$$p(y|\boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}\left(y|\boldsymbol{w}_{\boldsymbol{\mu}}^{\mathsf{T}} f(\boldsymbol{x}; \boldsymbol{w}_{\text{shared}}), \sigma_+(\boldsymbol{w}_{\boldsymbol{\sigma}}^{\mathsf{T}} f(\boldsymbol{x}; \boldsymbol{w}_{\text{shared}}))\right) \tag{13.21}$$

Figure 13.6 shows the advantage of this kind of model on a dataset where the mean grows linearly over time, with seasonal oscillations, and the variance increases quadratically. (This is a simple example of a **stochastic volatility model**; it can be used to model financial data, as well as the global temperature of the earth, which (due to climate change) is increasing in mean *and* in variance.) We see that a regression model where the output variance σ^2 is treated as a fixed (input-independent) parameter will sometimes be underconfident, since it needs to adjust to the overall noise level, and cannot adapt to the noise level at each point in input space.

13.2.5 The importance of depth

One can show that an MLP *with one hidden layer* is a **universal function approximator**, meaning it can model any suitably smooth function, given enough hidden units, to any desired level of accuracy [HSW89; Cyb89; Hor91]. Intuitively, the reason for this is that each hidden unit can specify a half plane, and a sufficiently large combination of these can "carve up" any region of space, to which we can associate any response (this is easiest to see when using piecewise linear activation functions, as shown in Figure 13.7).

However, various arguments, both experimental and theoretical (e.g., [Has87; Mon+14; Rag+17; Pog+17]), have shown that deep networks work better than shallow ones. The reason is that later layers can leverage the features that are learned by earlier layers; that is, the function is defined in a **compositional** or **hierarchical** way. For example, suppose we want to classify DNA strings, and the positive class is associated with the string `*AA??CGCG??AA*`, where ? is a wildcard denoting any single character, and * is a wildcard denoting any sequence of characters (possibly of length 0). Although we could fit this with a single hidden layer model, intuitively it will be easier to learn if the model first learns to detect the AA and CG "motifs" using the hidden units in layer 1, and then uses



(a) (b)

Figure 13.7: A decomposition of $\mathbb{R}^2$ into a finite set of linear decision regions produced by an MLP with ReLU *activations with (a) one hidden layer of 25 hidden units and (b) two hidden layers. From Figure 1 of [HAB19]. Used with kind permission of Maksym Andriuschenko.*

these features to define a simple linear classifier in layer 2, analogously to how we solved the XOR problem in Section 13.2.1.

13.2.6 The "deep learning revolution"

Although the ideas behind DNNs date back several decades, it was not until the 2010s that they started to become very widely used. The first area to adopt these methods was the field of automatic speech recognition (ASR), based on breakthrough results in [Dah+11]. This approach rapidly became the standard paradigm, and was widely adopted in academia and industry [Hin+12].

However, the moment that got the most attention was when [KSH12] showed that deep CNNs could significantly improve performance on the challenging ImageNet image classification benchmark, reducing the error rate from 26% to 16% in a single year (see Figure 1.14b); this was a huge jump compared to the previous rate of progress of about 2% reduction per year.

The "explosion" in the usage of DNNs has several contributing factors. One is the availability of cheap **GPUs** (graphics processing units); these were originally developed to speed up image rendering for video games, but they can also massively reduce the time it takes to fit large CNNs, which involve similar kinds of matrix-vector computations. Another is the growth in large labeled datasets, which enables us to fit complex function approximators with many parameters without overfitting. (For example, ImageNet has 1.3M labeled images, and is used to fit models that have millions of parameters.) Indeed, if deep learning systems are viewed as "rockets", then large datasets have been called the fuel.[3]

Motivated by the outstanding empirical success of DNNs, various companies started to become interested in this technology. This had led to the development of high quality open-source software libraries, such as Tensorflow (made by Google), PyTorch (made by Facebook), and MXNet (made by Amazon). These libraries support automatic differentiation (see Section 13.3) and scalable gradient-based optimization (see Section 8.4) of complex differentiable functions. We will use some

3. This popular analogy is due to Andrew Ng, who mentioned it in a keynote talk at the GPU Technology Conference (GTC) in 2015. His slides are available at https://bit.ly/38RTxzH.

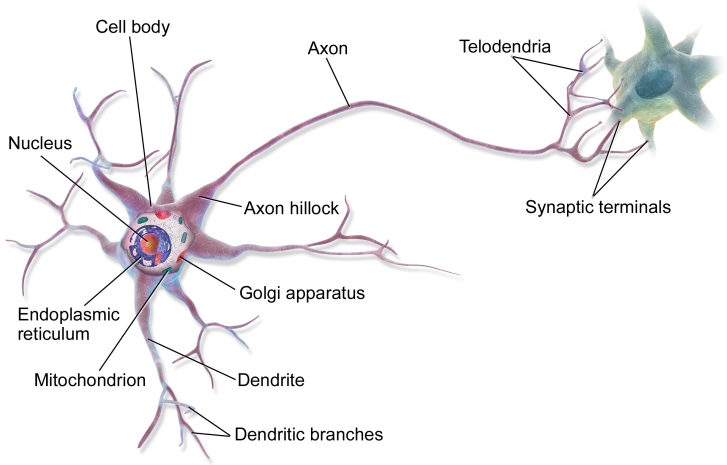


Figure 13.8: Illustration of two neurons connected together in a "circuit". The output axon of the left neuron makes a synaptic connection with the dendrites of the cell on the right. Electrical charges, in the form of ion flows, allow the cells to communicate. From https://en.wikipedia.org/wiki/Neuron. Used with kind permission of Wikipedia author BruceBlaus.

of these libraries in various places throughout the book to implement a variety of models, not just DNNs.[4]

More details on the history of the "deep learning revolution" can be found in e.g., [Sej18; Met21].

13.2.7 Connections with biology

In this section, we discuss the connections between the kinds of neural networks we have discussed above, known as **artificial neural networks** or **ANN**s, and real neural networks. The details on how real biological brains work are quite complex (see e.g., [Kan+12]), but we can give a simple "cartoon".

We start by considering a model of a single neuron. To a first approximation, we can say that whether neuron k fires, denoted by $h_k \in \{0, 1\}$, depends on the activity of its inputs, denoted by $\boldsymbol{x} \in \mathbb{R}^D$, as well as the strength of the incoming connections, which we denote by $\boldsymbol{w}_k \in \mathbb{R}^D$. We can compute a weighted sum of the inputs using $a_k = \boldsymbol{w}_k^\intercal \boldsymbol{x}$. These weights can be viewed as "wires" connecting the inputs x_d to neuron h_k; these are analogous to **dendrites** in a real neuron (see Figure 13.8). This weighted sum is then compared to a threshold, b_k, and if the activation exceeds the threshold, the neuron fires; this is analogous to the neuron emitting an electrical output or **action potential**. Thus we can model the behavior of the neuron using $h_k(\boldsymbol{x}) = H(\boldsymbol{w}_k^\intercal \boldsymbol{x} - b_k)$, where $H(a) = \mathbb{I}(a > 0)$ is the Heaviside function. This is called the **McCulloch-Pitts model** of the neuron, and was proposed in 1943 [MP43].

We can combine multiple such neurons together to make an ANN. The result has sometimes been

4. Note, however, that some have argued (see e.g., [BI19]) that current libraries are too inflexible, and put too much emphasis on methods based on dense matrix-vector multiplication, as opposed to more general algorithmic primitives.

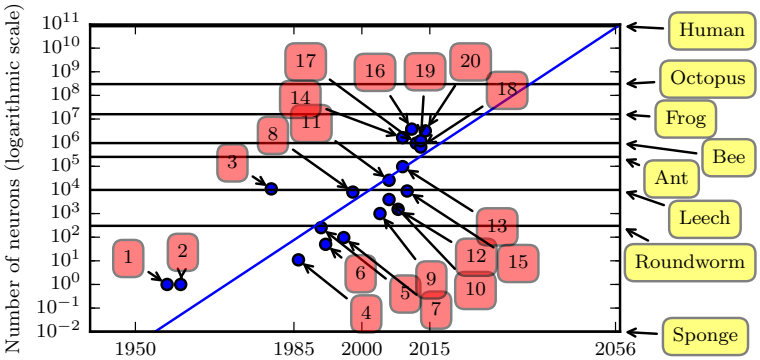


Figure 13.9: Plot of neural network sizes over time. Models 1, 2, 3 and 4 correspond to the perceptron [Ros58], the adaptive linear unit [WH60] the neocognitron [Fuk80], and the first MLP trained by backprop [RHW86]. Approximate number of neurons for some living organisms are shown on the right scale (the sponge has 0 neurons), based on https://en.wikipedia.org/wiki/List_of_animals_by_number_of_neurons. From Figure 1.11 of [GBC16]. Used with kind permission of Ian Goodfellow.

viewed as a model of the brain. However, ANNs differs from biological brains in many ways, including the following:

- Most ANNs use backpropagation to modify the strength of their connections (see Section 13.3). However, real brains do not use backprop, since there is no way to send information backwards along an axon [Ben+15b; BS16; KH19]. Instead, they use local update rules for adjusting synaptic strengths.
- Most ANNs are strictly feedforward, but real brains have many feedback connections. It is believed that this feedback acts like a prior, which can be combined with bottom up likelihoods from the sensory system to compute a posterior over hidden states of the world, which can then be used for optimal decision making (see e.g., [Doy+07]).
- Most ANNs use simplified neurons consisting of a weighted sum passed through a nonlinearity, but real biological neurons have complex dendritic tree structures (see Figure 13.8), with complex spatio-temporal dynamics.
- Most ANNs are smaller in size and number of connections than biological brains (see Figure 13.9). Of course, ANNs are getting larger every week, fueled by various new **hardware accelerators**, such as GPUs and **TPUs** (**tensor processing units**), etc. However, even if ANNs match biological brains in terms of number of units, the comparison is misleading since the processing capability of a biological neuron is much higher than an artificial neuron (see point above).
- Most ANNs are designed to model a single function, such as mapping an image to a label, or a sequence of words to another sequence of words. By contrast, biological brains are very complex systems, composed of multiple specialized interacting modules, which implement different kinds of functions or behaviors such as perception, control, memory, language, etc (see e.g., [Sha88; Kan+12]).

Of course, there are efforts to make realistic models of biological brains (e.g., the **Blue Brain Project** [Mar06; Yon19]). However, an interesting question is whether studying the brain at this level of detail is useful for "solving AI". It is commonly believed that the low level details of biological brains do not matter if our goal is to build "intelligent machines", just as aeroplanes do not flap their wings. However, presumably "AIs" will follow similar "laws of intelligence" to intelligent biological agents, just as planes and birds follow the same laws of aerodynamics.

Unfortunately, we do not yet know what the "laws of intelligence" are, or indeed if there even are such laws. In this book we make the assumption that any intelligent agent should follow the basic principles of information processing and Bayesian decision theory, which is known to be the optimal way to make decisions under uncertainty (see Section 5.1).

In practice, the optimal Bayesian approach is often computationally intractable. In the natural world, biological agents have evolved various algorithmic "shortcuts" to the optimal solution; this can explain many of the **heuristics** that people use in everyday reasoning [KST82; GTA00; Gri20]. As the tasks we want our machines to solve become harder, we may be able to gain insights from neuroscience and cognitive science for how to solve such tasks in an approximate way (see e.g., [SC16; MWK16; Has+17; Lak+17; HG21]). However, we should also bear in mind that AI/ML systems are increasingly used for safety-critical applications, in which we might want and expect the machine to do better than a human. In such cases, we may want more than just heuristic solutions that often work; instead we may want provably reliable methods, similar to other engineering fields (see Section 1.6.3 for further discussion).

13.3 Backpropagation

This section is coauthored with Mathieu Blondel.

In this section, we describe the famous **backpropagation algorithm**, which can be used to compute the gradient of a loss function applied to the output of the network wrt the parameters in each layer. This gradient can then be passed to a gradient-based optimization algorithm, as we discuss in Section 13.4.

The backpropagation algorithm was originally discovered in [BH69], and independently in [Wer74]. However, it was [RHW86] that brought the algorithm to the attention of the "mainstream" ML community. See the wikipedia page[5] for more historical details.

We initially assume the computation graph is a simple linear chain of stacked layers, as in an MLP. In this case, backprop is equivalent to repeated applications of the chain rule of calculus (see Equation (7.261)). However, the method can be generalized to arbitrary directed acyclic graphs (DAGs), as we discuss in Section 13.3.4. This general procedure is often called **automatic differentiation** or **autodiff**.

13.3.1 Forward vs reverse mode differentiation

Consider a mapping of the form $\boldsymbol{o} = \boldsymbol{f}(\boldsymbol{x})$, where $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{o} \in \mathbb{R}^m$. We assume that $\boldsymbol{f}$ is defined as a composition of functions:

$$\boldsymbol{f} = \boldsymbol{f}_4 \circ \boldsymbol{f}_3 \circ \boldsymbol{f}_2 \circ \boldsymbol{f}_1 \tag{13.22}$$

5. https://en.wikipedia.org/wiki/Backpropagation#History

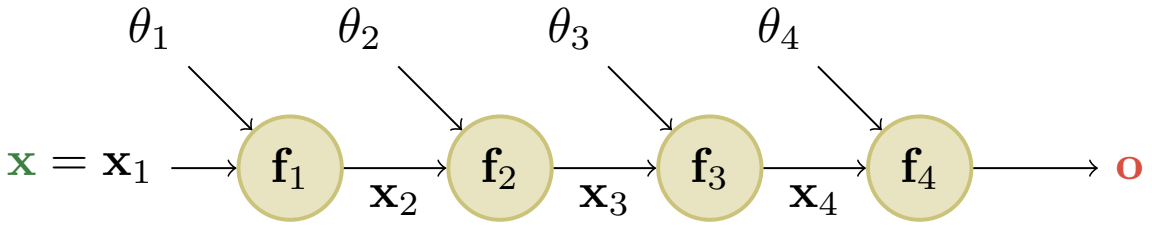


Figure 13.10: A simple linear-chain feedforward model with 4 layers. Here $\boldsymbol{x}$ is the input and $\boldsymbol{o}$ is the output. From [Blo20].

where $\boldsymbol{f}_1 : \mathbb{R}^n \to \mathbb{R}^{m_1}$, $\boldsymbol{f}_2 : \mathbb{R}^{m_1} \to \mathbb{R}^{m_2}$, $\boldsymbol{f}_3 : \mathbb{R}^{m_2} \to \mathbb{R}^{m_3}$, and $\boldsymbol{f}_4 : \mathbb{R}^{m_3} \to \mathbb{R}^m$. The intermediate steps needed to compute $\boldsymbol{o} = \boldsymbol{f}(\boldsymbol{x})$ are $\boldsymbol{x}_2 = \boldsymbol{f}_1(\boldsymbol{x})$, $\boldsymbol{x}_3 = \boldsymbol{f}_2(\boldsymbol{x}_2)$, $\boldsymbol{x}_4 = \boldsymbol{f}_3(\boldsymbol{x}_3)$, and $\boldsymbol{o} = \boldsymbol{f}_4(\boldsymbol{x}_4)$.

We can compute the Jacobian $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}} \in \mathbb{R}^{m \times n}$ using the chain rule:

$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_4}\frac{\partial \boldsymbol{x}_4}{\partial \boldsymbol{x}_3}\frac{\partial \boldsymbol{x}_3}{\partial \boldsymbol{x}_2}\frac{\partial \boldsymbol{x}_2}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}_4(\boldsymbol{x}_4)}{\partial \boldsymbol{x}_4}\frac{\partial \boldsymbol{f}_3(\boldsymbol{x}_3)}{\partial \boldsymbol{x}_3}\frac{\partial \boldsymbol{f}_2(\boldsymbol{x}_2)}{\partial \boldsymbol{x}_2}\frac{\partial \boldsymbol{f}_1(\boldsymbol{x})}{\partial \boldsymbol{x}} \tag{13.23}$$

$$= \mathbf{J}_{\boldsymbol{f}_4}(\boldsymbol{x}_4)\mathbf{J}_{\boldsymbol{f}_3}(\boldsymbol{x}_3)\mathbf{J}_{\boldsymbol{f}_2}(\boldsymbol{x}_2)\mathbf{J}_{\boldsymbol{f}_1}(\boldsymbol{x}) \tag{13.24}$$

We now discuss how to compute the Jacobian $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ efficiently. Recall that

$$\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}(\boldsymbol{x})}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^\mathsf{T} \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\mathsf{T} \end{pmatrix} = \left(\frac{\partial \boldsymbol{f}}{\partial x_1}, \cdots, \frac{\partial \boldsymbol{f}}{\partial x_n} \right) \in \mathbb{R}^{m \times n} \tag{13.25}$$

where $\nabla f_i(\boldsymbol{x})^\mathsf{T} \in \mathbb{R}^{1 \times n}$ is the i'th row (for $i = 1 : m$) and $\frac{\partial \boldsymbol{f}}{\partial x_j} \in \mathbb{R}^m$ is the j'th column (for $j = 1 : n$). Note that, in our notation, when $m = 1$, the gradient, denoted $\nabla \boldsymbol{f}(\boldsymbol{x})$, has the same shape as $\boldsymbol{x}$. It is therefore a column vector, while $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ is a row vector. In this case, we therefore technically have $\nabla \boldsymbol{f}(\boldsymbol{x}) = \mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})^\mathsf{T}$.

We can extract the i'th row from $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ by using a vector Jacobian product (VJP) of the form $\boldsymbol{e}_i^\mathsf{T}\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$, where $\boldsymbol{e}_i \in \mathbb{R}^m$ is the unit basis vector. Similarly, we can extract the j'th column from $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ by using a Jacobian vector product (JVP) of the form $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})\boldsymbol{e}_j$, where $\boldsymbol{e}_j \in \mathbb{R}^n$. This shows that the computation of $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ reduces to either n JVPs or m VJPs.

If $n < m$, it is more efficient to compute $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ for each column $j = 1 : n$ by using JVPs in a right-to-left manner. The right multiplication with a column vector $\boldsymbol{v}$ is

$$\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})\boldsymbol{v} = \underbrace{\mathbf{J}_{\boldsymbol{f}_4}(\boldsymbol{x}_4)}_{m \times m_3}\underbrace{\mathbf{J}_{\boldsymbol{f}_3}(\boldsymbol{x}_3)}_{m_3 \times m_2}\underbrace{\mathbf{J}_{\boldsymbol{f}_2}(\boldsymbol{x}_2)}_{m_2 \times m_1}\underbrace{\mathbf{J}_{\boldsymbol{f}_1}(\boldsymbol{x}_1)}_{m_1 \times n}\underbrace{\boldsymbol{v}}_{n \times 1} \tag{13.26}$$

This can be computed using **forward mode differentiation**; see Algorithm 13.1 for the pseudocode. Assuming $m = 1$ and $n = m_1 = m_2 = m_3$, the cost of computing $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ is $O(n^2)$.

If $n > m$ (e.g., if the output is a scalar), it is more efficient to compute $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ for each row $i = 1 : m$ by using VJPs in a left-to-right manner. The left multiplication with a row vector $\boldsymbol{u}^\mathsf{T}$ is

$$\boldsymbol{u}^\mathsf{T}\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \underbrace{\boldsymbol{u}^\mathsf{T}}_{1 \times m}\underbrace{\mathbf{J}_{\boldsymbol{f}_4}(\boldsymbol{x}_4)}_{m \times m_3}\underbrace{\mathbf{J}_{\boldsymbol{f}_3}(\boldsymbol{x}_3)}_{m_3 \times m_2}\underbrace{\mathbf{J}_{\boldsymbol{f}_2}(\boldsymbol{x}_2)}_{m_2 \times m_1}\underbrace{\mathbf{J}_{\boldsymbol{f}_1}(\boldsymbol{x}_1)}_{m_1 \times n} \tag{13.27}$$

Algorithm 13.1: Foward mode differentiation

$\boldsymbol{x}_1 := \boldsymbol{x}$
$\boldsymbol{v}_j := \boldsymbol{e}_j \in \mathbb{R}^n$ for $j = 1 : n$
for $k = 1 : K$ **do**
  $\boldsymbol{x}_{k+1} = \boldsymbol{f}_k(\boldsymbol{x}_k)$
  $\boldsymbol{v}_j := \mathbf{J}_{\boldsymbol{f}_k}(\boldsymbol{x}_k)\boldsymbol{v}_j$ for $j = 1 : n$
Return $\boldsymbol{o} = \boldsymbol{x}_{K+1}$, $[\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})]_{:,j} = \boldsymbol{v}_j$ for $j = 1 : n$

This can be done using **reverse mode differentiation**; see Algorithm 13.2 for the pseudocode. Assuming $m = 1$ and $n = m_1 = m_2 = m_3$, the cost of computing $\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ is $O(n^2)$.

Algorithm 13.2: Reverse mode differentiation

$\boldsymbol{x}_1 := \boldsymbol{x}$
for $k = 1 : K$ **do**
  $\boldsymbol{x}_{k+1} = \boldsymbol{f}_k(\boldsymbol{x}_k)$
$\boldsymbol{u}_i := \boldsymbol{e}_i \in \mathbb{R}^m$ for $i = 1 : m$
for $k = K : 1$ **do**
  $\boldsymbol{u}_i^\mathsf{T} := \boldsymbol{u}_i^\mathsf{T}\mathbf{J}_{\boldsymbol{f}_k}(\boldsymbol{x}_k)$ for $i = 1 : m$
Return $\boldsymbol{o} = \boldsymbol{x}_{K+1}$, $[\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})]_{i,:} = \boldsymbol{u}_i^\mathsf{T}$ for $i = 1 : m$

Both Algorithms 13.1 and 13.2 can be adapted to compute JVPs and VJPs against *any* collection of input vectors, by accepting $\{\boldsymbol{v}_j\}_{j=1,\ldots,n}$ and $\{\boldsymbol{u}_i\}_{i=1,\ldots,m}$ as respective inputs. Initializing these vectors to the standard basis is useful specifically for producing the complete Jacobian as output.

13.3.2 Reverse mode differentiation for multilayer perceptrons

In the previous section, we considered a simple linear-chain feedforward model where each layer does not have any learnable parameters. In this section, each layer can now have (optional) parameters $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_4$. See Figure 13.10 for an illustration. We focus on the case where the mapping has the form $\mathcal{L} : \mathbb{R}^n \to \mathbb{R}$, so the output is a scalar. For example, consider ℓ_2 loss for a MLP with one hidden layer:

$$\mathcal{L}((\boldsymbol{x}, \boldsymbol{y}), \boldsymbol{\theta}) = \frac{1}{2}||\boldsymbol{y} - \mathbf{W}_2\varphi(\mathbf{W}_1\boldsymbol{x})||_2^2 \tag{13.28}$$

we can represent this as the following feedforward model:

$$\mathcal{L} = \boldsymbol{f}_4 \circ \boldsymbol{f}_3 \circ \boldsymbol{f}_2 \circ \boldsymbol{f}_1 \tag{13.29}$$
$$\boldsymbol{x}_2 = \boldsymbol{f}_1(\boldsymbol{x}, \boldsymbol{\theta}_1) = \mathbf{W}_1 \boldsymbol{x} \tag{13.30}$$
$$\boldsymbol{x}_3 = \boldsymbol{f}_2(\boldsymbol{x}_2, \emptyset) = \varphi(\boldsymbol{x}_2) \tag{13.31}$$
$$\boldsymbol{x}_4 = \boldsymbol{f}_3(\boldsymbol{x}_3, \boldsymbol{\theta}_3) = \mathbf{W}_2 \boldsymbol{x}_3 \tag{13.32}$$
$$\mathcal{L} = \boldsymbol{f}_4(\boldsymbol{x}_4, \boldsymbol{y}) = \frac{1}{2}||\boldsymbol{x}_4 - \boldsymbol{y}||^2 \tag{13.33}$$

We use the notation $\boldsymbol{f}_k(\boldsymbol{x}_k, \boldsymbol{\theta}_k)$ to denote the function at layer k, where $\boldsymbol{x}_k$ is the previous output and $\boldsymbol{\theta}_k$ are the optional parameters for this layer.

In this example, the final layer returns a scalar, since it corresponds to a loss function $\mathcal{L} \in \mathbb{R}$. Therefore it is more efficient to use reverse mode differentation to compute the gradient vectors.

We first discuss how to compute the gradient of the scalar output wrt the parameters in each layer. We can easily compute the gradient wrt the predictions in the final layer $\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_4}$. For the gradient wrt the parameters in the earlier layers, we can use the chain rule to get

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_3} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_4} \frac{\partial \boldsymbol{x}_4}{\partial \boldsymbol{\theta}_3} \tag{13.34}$$
$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_2} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_4} \frac{\partial \boldsymbol{x}_4}{\partial \boldsymbol{x}_3} \frac{\partial \boldsymbol{x}_3}{\partial \boldsymbol{\theta}_2} \tag{13.35}$$
$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_4} \frac{\partial \boldsymbol{x}_4}{\partial \boldsymbol{x}_3} \frac{\partial \boldsymbol{x}_3}{\partial \boldsymbol{x}_2} \frac{\partial \boldsymbol{x}_2}{\partial \boldsymbol{\theta}_1} \tag{13.36}$$

where each $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_k} = (\nabla_{\boldsymbol{\theta}_k} \mathcal{L})^\mathsf{T}$ is a d_k-dimensional gradient row vector, where d_k is the number of parameters in layer k. We see that these can be computed recursively, by multiplying the gradient row vector at layer k by the Jacobian $\frac{\partial \boldsymbol{x}_k}{\partial \boldsymbol{x}_{k-1}}$ which is an $n_k \times n_{k-1}$ matrix, where n_k is the number of hidden units in layer k. See Algorithm 13.3 for the pseudocode.

This algorithm computes the gradient of the loss wrt the parameters at each layer. It also computes the gradient of the loss wrt the input, $\nabla_{\boldsymbol{x}} \mathcal{L} \in \mathbb{R}^n$, where n is the dimensionality of the input. This latter quantity is not needed for parameter learning, but can be useful for generating inputs to a model (see Section 14.6 for some applications).

All that remains is to specify how to compute the vector Jacobian product (VJP) of all supported layers. The details of this depend on the form of the function at each layer. We discuss some examples below.

13.3.3 Vector-Jacobian product for common layers

Recall that the Jacobian for a layer of the form $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$. is defined by

$$\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}(\boldsymbol{x})}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^\mathsf{T} \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\mathsf{T} \end{pmatrix} = \left(\frac{\partial \boldsymbol{f}}{\partial x_1}, \cdots, \frac{\partial \boldsymbol{f}}{\partial x_n} \right) \in \mathbb{R}^{m \times n} \tag{13.37}$$

where $\nabla f_i(\boldsymbol{x})^\mathsf{T} \in \mathbb{R}^n$ is the i'th row (for $i = 1 : m$) and $\frac{\partial \boldsymbol{f}}{\partial x_j} \in \mathbb{R}^m$ is the j'th column (for $j = 1 : n$). In this section, we describe how to compute the VJP $\boldsymbol{u}^\mathsf{T} \mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})$ for common layers.

Algorithm 13.3: Backpropagation for an MLP with K layers

// Forward pass
$\boldsymbol{x}_1 := \boldsymbol{x}$
for $k = 1 : K$ **do**
  $\boldsymbol{x}_{k+1} = \boldsymbol{f}_k(\boldsymbol{x}_k, \boldsymbol{\theta}_k)$
// Backward pass
$\boldsymbol{u}_{K+1} := 1$
for $k = K : 1$ **do**
  $\boldsymbol{g}_k := \boldsymbol{u}_{k+1}^\mathsf{T} \frac{\partial \boldsymbol{f}_k(\boldsymbol{x}_k, \boldsymbol{\theta}_k)}{\partial \boldsymbol{\theta}_k}$
  $\boldsymbol{u}_k^\mathsf{T} := \boldsymbol{u}_{k+1}^\mathsf{T} \frac{\partial \boldsymbol{f}_k(\boldsymbol{x}_k, \boldsymbol{\theta}_k)}{\partial \boldsymbol{x}_k}$
// Output
Return $\mathcal{L} = \boldsymbol{x}_{K+1}$, $\nabla_{\boldsymbol{x}} \mathcal{L} = \boldsymbol{u}_1$, $\{\nabla_{\boldsymbol{\theta}_k} \mathcal{L} = \boldsymbol{g}_k : k = 1 : K\}$

13.3.3.1 Cross entropy layer

Consider a cross-entropy loss layer taking logits $\boldsymbol{x}$ and target labels $\boldsymbol{y}$ as input, and returning a scalar:

$$z = f(\boldsymbol{x}) = \text{CrossEntropyWithLogits}(\boldsymbol{y}, \boldsymbol{x}) = -\sum_c y_c \log(\text{softmax}(\boldsymbol{x})_c) = -\sum_c y_c \log p_c \quad (13.38)$$

where $\boldsymbol{p} = \text{softmax}(\boldsymbol{x}) = \frac{e^{x_c}}{\sum_{c'=1}^C e^{x_{c'}}}$ are the predicted class probabilites, and $\boldsymbol{y}$ is the true distribution over labels (often a one-hot vector). The Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial z}{\partial \boldsymbol{x}} = (\boldsymbol{p} - \boldsymbol{y})^\mathsf{T} \in \mathbb{R}^{1 \times C} \quad (13.39)$$

To see this, assume the target label is class c. We have

$$z = f(\boldsymbol{x}) = -\log(p_c) = -\log\left(\frac{e^{x_c}}{\sum_j e^{x_j}}\right) = \log\left(\sum_j e^{x_j}\right) - x_c \quad (13.40)$$

Hence

$$\frac{\partial z}{\partial x_i} = \frac{\partial}{\partial x_i} \log \sum_j e^{x_j} - \frac{\partial}{\partial x_i} x_c = \frac{e^{x_i}}{\sum_j e^{x_j}} - \frac{\partial}{\partial x_i} x_c = p_i - \mathbb{I}(i = c) \quad (13.41)$$

If we define $\boldsymbol{y} = [\mathbb{I}(i = c)]$, we recover Equation (13.39). Note that the Jacobian of this layer is a row vector, since the output is a scalar.

13.3.3.2 Elementwise nonlinearity

Consider a layer that applies an elementwise nonlinearity, $\boldsymbol{z} = \boldsymbol{f}(\boldsymbol{x}) = \varphi(\boldsymbol{x})$, so $z_i = \varphi(x_i)$. The (i, j) element of the Jacobian is given by

$$\frac{\partial z_i}{\partial x_j} = \begin{cases} \varphi'(x_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{13.42}$$

where $\varphi'(a) = \frac{d}{da}\varphi(a)$. In other words, the Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \text{diag}(\varphi'(\boldsymbol{x})) \tag{13.43}$$

For an arbitrary vector $\boldsymbol{u}$, we can compute $\boldsymbol{u}^\mathsf{T}\mathbf{J}$ by elementwise multiplication of the diagonal elements of $\mathbf{J}$ with $\boldsymbol{u}$. For example, if

$$\varphi(a) = \text{ReLU}(a) = \max(a, 0) \tag{13.44}$$

we have

$$\varphi'(a) = \begin{cases} 0 & a < 0 \\ 1 & a > 0 \end{cases} \tag{13.45}$$

The subderivative (Section 8.1.4.1) at $a = 0$ is any value in $[0, 1]$. It is often taken to be 0. Hence

$$\text{ReLU}'(a) = H(a) \tag{13.46}$$

where H is the Heaviside step function.

13.3.3.3 Linear layer

Now consider a linear layer, $\boldsymbol{z} = \boldsymbol{f}(\boldsymbol{x}, \mathbf{W}) = \mathbf{W}\boldsymbol{x}$, where $\mathbf{W} \in \mathbb{R}^{m \times n}$, so $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{z} \in \mathbb{R}^m$. We can compute the Jacobian wrt the input vector, $\mathbf{J} = \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} \in \mathbb{R}^{m \times n}$, as follows. Note that

$$z_i = \sum_{k=1}^{n} W_{ik} x_k \tag{13.47}$$

So the (i, j) entry of the Jacobian will be

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_{k=1}^{n} W_{ik} x_k = \sum_{k=1}^{n} W_{ik} \frac{\partial}{\partial x_j} x_k = W_{ij} \tag{13.48}$$

since $\frac{\partial}{\partial x_j} x_k = \mathbb{I}(k = j)$. Hence the Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \mathbf{W} \tag{13.49}$$

The VJP between $\boldsymbol{u}^\mathsf{T} \in \mathbb{R}^{1\times m}$ and $\mathbf{J} \in \mathbb{R}^{m\times n}$ is

$$\boldsymbol{u}^\mathsf{T} \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \boldsymbol{u}^\mathsf{T} \mathbf{W} \in \mathbb{R}^{1\times n} \tag{13.50}$$

Now consider the Jacobian wrt the weight matrix, $\mathbf{J} = \frac{\partial \boldsymbol{z}}{\partial \mathbf{W}}$. This can be represented as a $m \times (m \times n)$ matrix, which is complex to deal with. So instead, let us focus on taking the gradient wrt a single weight, W_{ij}. This is easier to compute, since $\frac{\partial \boldsymbol{z}}{\partial W_{ij}}$ is a vector. To compute this, note that

$$z_k = \sum_{l=1}^{n} W_{kl} x_l \tag{13.51}$$

$$\frac{\partial z_k}{\partial W_{ij}} = \sum_{l=1}^{n} x_l \frac{\partial}{\partial W_{ij}} W_{kl} = \sum_{l=1}^{n} x_l \mathbb{I}\left(i = k \text{ and } j = l\right) \tag{13.52}$$

Hence

$$\frac{\partial \boldsymbol{z}}{\partial W_{ij}} = \begin{pmatrix} 0 & \cdots & 0 & x_j & 0 & \cdots & 0 \end{pmatrix}^\mathsf{T} \tag{13.53}$$

where the non-zero entry occurs in location i. The VJP between $\boldsymbol{u}^\mathsf{T} \in \mathbb{R}^{1\times m}$ and $\frac{\partial \boldsymbol{z}}{\partial \mathbf{W}} \in \mathbb{R}^{m\times(m\times n)}$ can be represented as a matrix of shape $1 \times (m \times n)$. Note that

$$\boldsymbol{u}^\mathsf{T} \frac{\partial \boldsymbol{z}}{\partial W_{ij}} = \sum_{k=1}^{m} u_k \frac{\partial z_k}{\partial W_{ij}} = u_i x_j \tag{13.54}$$

Therefore

$$\left[\boldsymbol{u}^\mathsf{T} \frac{\partial \boldsymbol{z}}{\partial \mathbf{W}}\right]_{1,:} = \boldsymbol{u}\boldsymbol{x}^\mathsf{T} \in \mathbb{R}^{m\times n} \tag{13.55}$$

13.3.3.4 Putting it all together

For an exercise that puts this all together, see Exercise 13.1.

13.3.4 Computation graphs

MLPs are a simple kind of DNN in which each layer feeds directly into the next, forming a chain structure, as shown in Figure 13.10. However, modern DNNs can combine differentiable components in much more complex ways, to create a **computation graph**, analogous to how programmers combine elementary functions to make more complex ones. (Indeed, some have suggested that "deep learning" be called "**differentiable programming**".) The only restriction is that the resulting computation graph corresponds to a **directed acyclic graph** (**DAG**), where each node is a differentiable function of all its inputs.

For example, consider the function

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_1}} \tag{13.56}$$

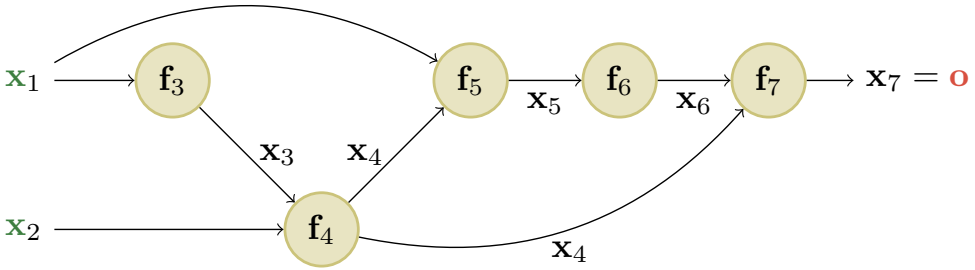


Figure 13.11: An example of a computation graph with 2 (scalar) inputs and 1 (scalar) output. From [Blo20].

We can compute this using the DAG in Figure 13.11, with the following intermediate functions:

$$x_3 = f_3(x_1) = e^{x_1} \tag{13.57}$$
$$x_4 = f_4(x_2, x_3) = x_2 x_3 \tag{13.58}$$
$$x_5 = f_5(x_1, x_4) = x_1 + x_4 \tag{13.59}$$
$$x_6 = f_6(x_5) = \sqrt{x_5} \tag{13.60}$$
$$x_7 = f_7(x_4, x_6) = x_4 x_6 \tag{13.61}$$

Note that we have numbered the nodes in topological order (parents before children). During the backward pass, since the graph is no longer a chain, we may need to sum gradients along multiple paths. For example, since x_4 influences x_5 and x_7, we have

$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_4} = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_5}\frac{\partial \boldsymbol{x}_5}{\partial \boldsymbol{x}_4} + \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_7}\frac{\partial \boldsymbol{x}_7}{\partial \boldsymbol{x}_4} \tag{13.62}$$

We can avoid repeated computation by working in reverse topological order. For example,

$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_7} = \frac{\partial \boldsymbol{x}_7}{\partial \boldsymbol{x}_7} = \mathbf{I}_m \tag{13.63}$$
$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_6} = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_7}\frac{\partial \boldsymbol{x}_7}{\partial \boldsymbol{x}_6} \tag{13.64}$$
$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_5} = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_6}\frac{\partial \boldsymbol{x}_6}{\partial \boldsymbol{x}_5} \tag{13.65}$$
$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_4} = \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_5}\frac{\partial \boldsymbol{x}_5}{\partial \boldsymbol{x}_4} + \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_7}\frac{\partial \boldsymbol{x}_7}{\partial \boldsymbol{x}_4} \tag{13.66}$$

In general, we use

$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_j} = \sum_{k \in \mathrm{Ch}(j)} \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_k}\frac{\partial \boldsymbol{x}_k}{\partial \boldsymbol{x}_j} \tag{13.67}$$

where the sum is over all children k of node j, as shown in Figure 13.12. The $\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{x}_k}$ gradient vector has already been computed for each child k; this quantity is called the **adjoint**. This gets multiplied by the Jacobian $\frac{\partial \boldsymbol{x}_k}{\partial \boldsymbol{x}_j}$ of each child.

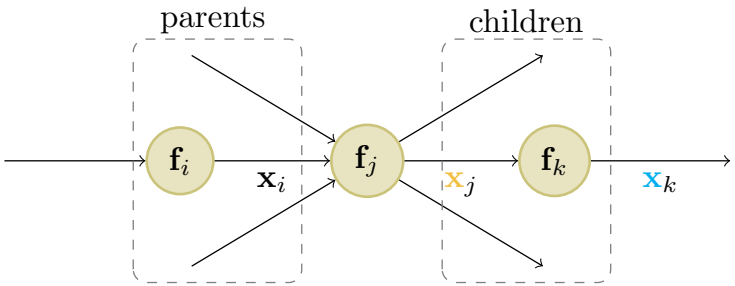


Figure 13.12: Notation for automatic differentiation at node j in a computation graph. From [Blo20].

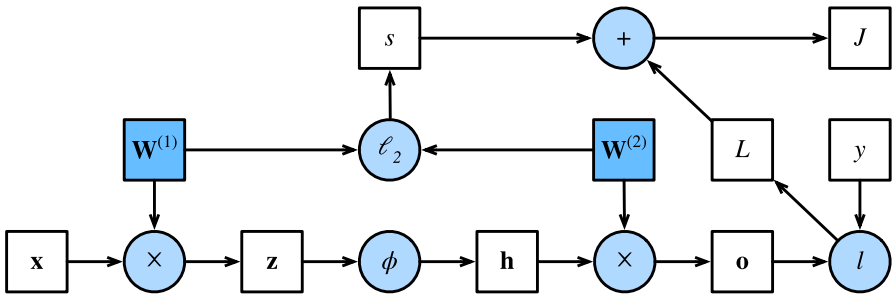


Figure 13.13: Computation graph for an MLP with input $\boldsymbol{x}$, hidden layer $\boldsymbol{h}$, output $\boldsymbol{o}$, loss function $L = \ell(\boldsymbol{o}, y)$, an ℓ_2 regularizer s on the weights, and total loss $J = L + s$. From Figure 4.7.1 of [Zha+20]. Used with kind permission of Aston Zhang.

The computation graph can be computed ahead of time, by using an API to define a **static graph**. (This is how Tensorflow 1 worked.) Alternatively, the graph can be computed "**just in time**", by **tracing** the execution of the function on an input argument. (This is how Tensorflow eager mode works, as well as JAX and PyTorch.) The latter approach makes it easier to work with a **dynamic graph**, whose shape can change depending on the values computed by the function.

Figure 13.13 shows a computation graph corresponding to an MLP with one hidden layer with weight decay. More precisely, the model computes the linear pre-activations $\boldsymbol{z} = \mathbf{W}^{(1)}\boldsymbol{x}$, the hidden activations $\boldsymbol{h} = \phi(\boldsymbol{z})$, the linear outputs $\boldsymbol{o} = \mathbf{W}^{(2)}\boldsymbol{h}$, the loss $L = \ell(\boldsymbol{o}, y)$, the regularizer $s = \frac{\lambda}{2}(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2)$, and the total loss $J = L + s$.

13.4 Training neural networks

In this section, we discuss how to fit DNNs to data. The standard approach is to use maximum likelihood estimation, by minimizing the NLL:

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(\boldsymbol{y}_n|\boldsymbol{x}_n;\boldsymbol{\theta}) \tag{13.68}$$

It is also common to add a regularizer (such as the negative log prior), as we discuss in Section 13.5.

In principle we can just use the backprop algorithm (Section 13.3) to compute the gradient of this loss and pass it to an off-the-shelf optimizer, such as those discussed in Chapter 8. (The Adam optimizer of Section 8.4.6.3 is a popular choice, due to its ability to scale to large datasets (by virtue of being an SGD-type algorithm), and to converge fairly quickly (by virtue of using diagonal preconditioning and momentum).) However, in practice this may not work well. In this section, we discuss various problems that may arise, as well as some solutions. For more details on the practicalities of training DNNs, see various other books, such as [HG20; Zha+20; Gér19].

In addition to practical issues, there are important theoretical issues. In particular, we note that the DNN loss is not a convex objective, so in general we will not be able to find the global optimum. Nevertheless, SGD can often find suprisingly good solutions. The research into why this is the case is still being conducted; see [Bah+20] for a recent review of some of this work.

13.4.1 Tuning the learning rate

It is important to tune the learning rate (step size), to ensure convergence to a good solution. We discuss this issue in Section 8.4.3.

13.4.2 Vanishing and exploding gradients

When training very deep models, the gradient tends to become either very small (this is called the **vanishing gradient problem**) or very large (this is called the **exploding gradient problem**), because the error signal is being passed through a series of layers which either amplify or diminish it [Hoc+01]. (Similar problems arise in RNNs on long sequences, as we explain in Section 15.2.6.)

To explain the problem in more detail, consider the gradient of the loss wrt a node at layer l:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_l} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_{l+1}} \frac{\partial \boldsymbol{z}_{l+1}}{\partial \boldsymbol{z}_l} = \mathbf{J}_l \boldsymbol{g}_{l+1} \tag{13.69}$$

where $\mathbf{J}_l = \frac{\partial \boldsymbol{z}_{l+1}}{\partial \boldsymbol{z}_l}$ is the Jacobian matrix, and $\boldsymbol{g}_{l+1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_{l+1}}$ is the gradient at the next layer. If $\mathbf{J}_l$ is constant across layers, it is clear that the contribution of the gradient from the final layer, $\boldsymbol{g}_L$, to layer l will be $\mathbf{J}^{L-l}\boldsymbol{g}_L$. Thus the behavior of the system depends on the eigenvectors of $\mathbf{J}$.

Although $\mathbf{J}$ is a real-valued matrix, it is not (in general) symmetric, so its eigenvalues and eigenvectors can be complex-valued, with the imaginary components corresponding to oscillatory behavior. Let λ be the **spectral radius** of $\mathbf{J}$, which is the maximum of the absolute values of the eigenvalues. If this is greater than 1, the gradient can explode; if this is less than 1, the gradient can vanish. (Similarly, the spectral radius of $\mathbf{W}$, connecting $\boldsymbol{z}_l$ to $\boldsymbol{z}_{l+1}$, determines the stability of the dynamical system when run in forwards mode.)

The exploding gradient problem can be ameliorated by **gradient clipping**, in which we cap the magnitude of the gradient if it becomes too large, i.e., we use

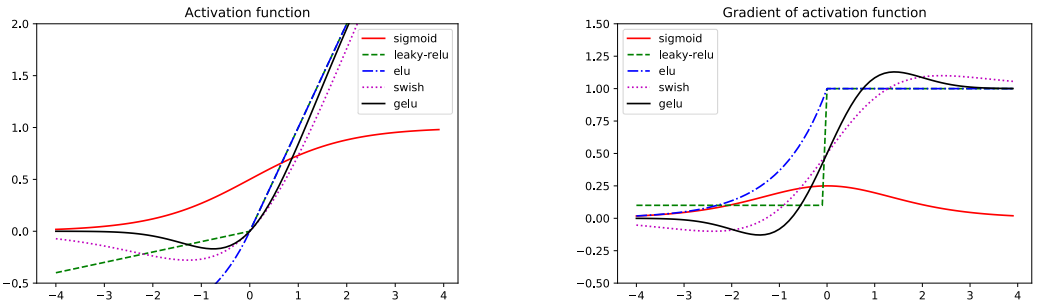
$$\boldsymbol{g}' = \min(1, \frac{c}{||\boldsymbol{g}||})\boldsymbol{g} \tag{13.70}$$

This way, the norm of $\boldsymbol{g}'$ can never exceed c, but the vector is always in the same direction as $\boldsymbol{g}$.

However, the vanishing gradient problem is more difficult to solve. There are various solutions, such as the following:

Name	Definition	Range	Reference
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$[0, 1]$	
Hyperbolic tangent	$\tanh(a) = 2\sigma(2a) - 1$	$[-1, 1]$	
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$	[GBB11]
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$	[GBB11; KSH12]
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$	[MHN13]
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$	[CUH16]
Swish	$a\sigma(a)$	$[-\infty, \infty]$	[RZL17]
GELU	$a\Phi(a)$	$[-\infty, \infty]$	[HG16]

Table 13.4: List of some popular activation functions for neural networks.



(a) *(b)*

Figure 13.14: (a) Some popular activation functions. (b) Plot of their gradients. Generated by activation_fun_deriv_jax.ipynb.

- Modify the the activation functions at each layer to prevent the gradient from becoming too large or too small; see Section 13.4.3.
- Modify the architecture so that the updates are additive rather than multiplicative; see Section 13.4.4.
- Modify the architecture to standardize the activations at each layer, so that the distribution of activations over the dataset remains constant during training; see Section 14.2.4.1.
- Carefully choose the initial values of the parameters; see Section 13.4.5.

13.4.3 Non-saturating activation functions

In Section 13.2.3, we mentioned that the sigmoid activation function saturates at 0 for large negative inputs, and at 1 for large positive inputs. It turns out that the gradient signal in these regimes is 0, preventing backpropagation from working.

To see why the gradient vanishes, consider a layer which computes $\boldsymbol{z} = \sigma(\mathbf{W}\boldsymbol{x})$, where the activation

function is sigmoidal:

$$\varphi(a) = \sigma(a) = \frac{1}{1 + \exp(-a)} \tag{13.71}$$

If the weights are initialized to be large (positive or negative), then it becomes very easy for $\boldsymbol{a} = \mathbf{W}\boldsymbol{x}$ to take on large values, and hence for $\boldsymbol{z}$ to saturate near $\mathbf{0}$ or $\mathbf{1}$, since the sigmoid saturates, as shown in Figure 13.14a.

Now let us consider how gradients propagate through this layer. The derivative of the activation function (elementwise) is given by

$$\varphi'(a) = \sigma(a)(1 - \sigma(a)) \tag{13.72}$$

See Figure 13.14b for a plot. By the chain rule, we can compute the Jacobian for this layer wrt the inputs by multiplying the Jacobian for the sigmoid nonlinearity, $\boldsymbol{z} = \sigma(\boldsymbol{a})$, using Equation (13.43), with the Jacobian for the linear layer, $\boldsymbol{a} = \mathbf{W}\boldsymbol{x}$, using Equation (13.49), to get

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \text{diag}(\boldsymbol{z}(1 - \boldsymbol{z})^\mathsf{T})\mathbf{W} \tag{13.73}$$

Hence, if $\boldsymbol{z}$ is near 0 or 1, the gradients wrt the inputs will go to 0.

Similarly, the Jacobian of this layer wrt the parameters can be computed by multiplying Equation (13.43) with Equation (13.55) to get

$$\frac{\partial \boldsymbol{z}}{\partial \mathbf{W}} = \boldsymbol{z}(1 - \boldsymbol{z})\boldsymbol{x}^\mathsf{T} \tag{13.74}$$

Hence, if $\boldsymbol{z}$ is near 0 or 1, the gradients wrt the parameters will go to 0.

One of the keys to being able to train very deep models is to use **non-saturating activation functions**. Several different functions have been proposed: see Table 13.4 for a summary.

13.4.3.1 ReLU

The most common is **rectified linear unit** or **ReLU**, proposed in [GBB11; KSH12]. This is defined as

$$\text{ReLU}(a) = \max(a, 0) = a\mathbb{I}(a > 0) \tag{13.75}$$

The ReLU function simply "turns off" negative inputs, and passes positive inputs unchanged. The gradient has the following form:

$$\text{ReLU}'(a) = \mathbb{I}(a > 0) \tag{13.76}$$

Now suppose we use this in a layer to compute $\boldsymbol{z} = \text{ReLU}(\mathbf{W}\boldsymbol{x})$. Using the results from Section 13.3.3, we can show that the Jacobian for this layer wrt the inputs has the form

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \mathbf{W}^\mathsf{T}\mathbb{I}(\boldsymbol{z} > \mathbf{0}) \tag{13.77}$$

and wrt the parameters has the form

$$\frac{\partial \boldsymbol{z}}{\partial \mathbf{W}} = \mathbb{I}(\boldsymbol{z} > \mathbf{0})\, \boldsymbol{x}^{\mathsf{T}} \tag{13.78}$$

Hence the gradient will not vanish, as long as $\boldsymbol{z}$ is positive.

Unfortunately, when using ReLU, if the weights are initialized to be large and negative, then it becomes very easy for (some components of) $\boldsymbol{a} = \mathbf{W}\boldsymbol{x}$ to take on large negative values, and hence for $\boldsymbol{z}$ to go to 0. This will cause the gradient for the weights to go to 0. The algorithm will never be able to escape this situation, so the hidden units (components of $\boldsymbol{z}$) will stay permanently off. This is called the "**dead** ReLU" problem [Lu+19].

13.4.3.2 Non-saturating ReLU

The problem of dead ReLU's can be solved by using non-saturating variants of ReLU. One alternate is the **leaky ReLU**, proposed in [MHN13]. This is defined as

$$\text{LReLU}(a; \alpha) = \max(\alpha a, a) \tag{13.79}$$

where $0 < \alpha < 1$. The slope of this function is 1 for positive inputs, and α for negative inputs, thus ensuring there is some signal passed back to earlier layers, even when the input is negative. See Figure 13.14b for a plot. If we allow the parameter α to be learned, rather than fixed, the leaky ReLU is called **parametric ReLU** [He+15].

Another popular choice is the **ELU**, proposed in [CUH16]. This is defined by

$$\text{ELU}(a; \alpha) = \begin{cases} \alpha(e^a - 1) & \text{if } a \leq 0 \\ a & \text{if } a > 0 \end{cases} \tag{13.80}$$

This has the advantage over leaky ReLU of being a smooth function.[6] See Figure 13.14 for plot.

A slight variant of ELU, known as **SELU** (self-normalizing ELU), was proposed in [Kla+17]. This has the form

$$\text{SELU}(a; \alpha, \lambda) = \lambda \text{ELU}(a; \alpha) \tag{13.81}$$

Surprisingly, they prove that by setting α and λ to carefully chosen values, this activation function is guaranteed to ensure that the output of each layer is standardized (provided the input is also standardized), even without the use of techniques such as batchnorm (Section 14.2.4.1). This can help with model fitting.

13.4.3.3 Other choices

As an alternative to manually discovering good activation functions, we can use blackbox optimization methods to search over the space of functional forms. Such an approach was used in [RZL17], where they discovered a function they call **swish** that seems to do well on some image classification benchmarks. It is defined by

$$\text{swish}(a; \beta) = a\sigma(\beta a) \tag{13.82}$$

6. ELU only has a continuous first derivative if $\alpha = 1$.

(The same function, under the name **SiLU** (for Sigmoid Linear Unit), was independently proposed in [HG16].) See Figure 13.14 for plot.

Another popular activation function is **GELU**, which stands for "Gaussian Error Linear Unit" [HG16]. This is defined as follows:

$$\text{GELU}(a) = a\Phi(a) \tag{13.83}$$

where $\Phi(a)$ is the cdf of a standard normal:

$$\Phi(a) = \Pr(\mathcal{N}(0,1) \leq a) = \frac{1}{2}\left(1 + \text{erf}(a/\sqrt{2})\right) \tag{13.84}$$

We see from Figure 13.14 that this is not a convex or monontonic function, unlike most other activation functions.

We can think of GELU as a "soft" version of ReLU, since it replaces the step function $\mathbb{I}(a > 0)$ with the Gaussian cdf, $\Phi(a)$. Alternatively, the GELU can be motivated as an adaptive version of dropout (Section 13.5.4), where we multiply the input by a binary scalar mask, $m \sim \text{Ber}(\Phi(a))$, where the probability of being dropped is given by $1 - \Phi(a)$. Thus the expected output is

$$\mathbb{E}[a] = \Phi(a) \times a + (1 - \Phi(a)) \times 0 = a\Phi(a) \tag{13.85}$$

We can approximate GELU using swish with a particular parameter setting, namely

$$\text{GELU}(a) \approx a\sigma(1.702a) \tag{13.86}$$

The **SmelU** activation function proposed in [SLC20] is another smooth version of ReLU but which results in more reproducibility in terms of training runs, due to enhanced robustness to numerical issues.

13.4.4 Residual connections

One solution to the vanishing gradient problem for DNNs is to use a **residual network** or **ResNet** [He+16a]. This is a feedforward model in which each layer has the form of a **residual block**, defined by

$$\mathcal{F}'_l(\boldsymbol{x}) = \mathcal{F}_l(\boldsymbol{x}) + \boldsymbol{x} \tag{13.87}$$

where $\mathcal{F}_l$ is a standard shallow nonlinear mapping (e.g., linear-activation-linear). The inner $\mathcal{F}_l$ function computes the residual term or delta that needs to be added to the input $\boldsymbol{x}$ to generate the desired output; it is often easier to learn to generate a small perturbation to the input than to directly predict the output. (Residual connections are usually used in conjunction with CNNs, as discussed in Section 14.3.4, but can also be used in MLPs.)

A model with residual connections has the same number of parameters as a model without residual connections, but it is easier to train. The reason is that gradients can flow directly from the output to earlier layers, as sketched in Figure 13.15b. To see this, note that the activations at the output layer can be derived in terms of any previous layer l using

$$\boldsymbol{z}_L = \boldsymbol{z}_l + \sum_{i=l}^{L-1} \mathcal{F}_i(\boldsymbol{z}_i; \boldsymbol{\theta}_i). \tag{13.88}$$

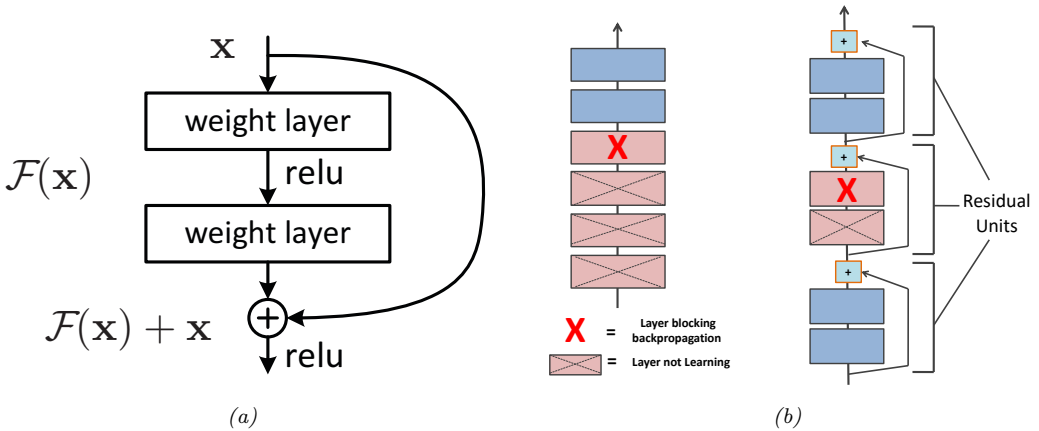


Figure 13.15: (a) Illustration of a residual block. (b) Illustration of why adding residual connections can help when training a very deep model. Adapted from Figure 14.16 of [Gér19].

We can therefore compute the gradient of the loss wrt the parameters of the l'th layer as follows:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_l} = \frac{\partial \boldsymbol{z}_l}{\partial \boldsymbol{\theta}_l}\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_l} \tag{13.89}$$

$$= \frac{\partial \boldsymbol{z}_l}{\partial \boldsymbol{\theta}_l}\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_L}\frac{\partial \boldsymbol{z}_L}{\partial \boldsymbol{z}_l} \tag{13.90}$$

$$= \frac{\partial \boldsymbol{z}_l}{\partial \boldsymbol{\theta}_l}\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_L}\left(1 + \sum_{i=l}^{L-1}\frac{\partial \mathcal{F}_i(\boldsymbol{z}_i;\boldsymbol{\theta}_i)}{\partial \boldsymbol{z}_l}\right) \tag{13.91}$$

$$= \frac{\partial \boldsymbol{z}_l}{\partial \boldsymbol{\theta}_l}\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_L} + \text{other terms} \tag{13.92}$$

Thus we see that the gradient at layer l depends directly on the gradient at layer L in a way that is independent of the depth of the network.

13.4.5 Parameter initialization

Since the objective function for DNN training is non-convex, the way that we initialize the parameters of a DNN can play a big role on what kind of solution we end up with, as well as how easy the function is to train (i.e., how well information can flow forwards and backwards through the model). In the rest of this section, we present some common heuristic methods that are used for initializing parameters.

13.4.5.1 Heuristic initialization schemes

In [GB10], they show that sampling parameters from a standard normal with fixed variance can result in exploding activations or gradients. To see why, consider a linear unit with no activation

function given by $o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j$; suppose $w_{ij} \sim \mathcal{N}(0, \sigma^2)$, and $\mathbb{E}[x_j] = 0$ and $\mathbb{V}[x_j] = \gamma^2$, where we assume x_j are independent of w_{ij}. The mean and variance of the output is given by

$$\mathbb{E}[o_i] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij} x_j] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}]\,\mathbb{E}[x_j] = 0 \tag{13.93}$$

$$\mathbb{V}[o_i] = \mathbb{E}\left[o_i^2\right] - (\mathbb{E}[o_i])^2 = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}\left[w_{ij}^2 x_j^2\right] - 0 = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}\left[w_{ij}^2\right]\mathbb{E}\left[x_j^2\right] = n_{\text{in}}\sigma^2\gamma^2 \tag{13.94}$$

To keep the output variance from blowing up, we need to ensure $n_{\text{in}}\sigma^2 = 1$ (or some other constant), where n_{in} is the **fan-in** of a unit (number of incoming connections).

Now consider the backwards pass. By analogous reasoning, we see that the variance of the gradients can blow up unless $n_{\text{out}}\sigma^2 = 1$, where n_{out} is the **fan-out** of a unit (number of outgoing connections). To satisfy both requirements at once, we set $\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1$, or equivalently

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}} \tag{13.95}$$

This is known as **Xavier initialization** or **Glorot initialization**, named after the first author of [GB10].

A special case arises if we use $\sigma^2 = 1/n_{\text{in}}$; this is known as **LeCun initialization**, named after Yann LeCun, who proposed it in the 1990s. This is equivalent to Glorot initialization when $n_{\text{in}} = n_{\text{out}}$. If we use $\sigma^2 = 2/n_{\text{in}}$, the method is called **He initialization**, named after Kaiming He, who proposed it in [He+15].

Note that it is not necessary to use a Gaussian distribution. Indeed, the above derivation just worked in terms of the first two moments (mean and variance), and made no assumptions about Gaussianity. For example, suppose we sample weights from a uniform distribution, $w_{ij} \sim \text{Unif}(-a, a)$. The mean is 0, and the variance is $\sigma^2 = a^2/3$. Hence we should set $a = \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}$.

Although the above derivation assumes a linear output unit, the technique works well empirically even for nonlinear units. The best choice of initialization method depends on which activation function you use. For linear, tanh, logistic, and softmax, Glorot is recommended. For ReLU and variants, He is recommended. For SELU, LeCun is recommended. See e.g., [Gér19] for more heuristics, and e.g., [HDR19] for some theory.

13.4.5.2 Data-driven initializations

We can also adopt a data-driven approach to parameter initialization. For example, [MM16] proposed a simple but effective scheme known as **layer-sequential unit-variance** (LSUV) initialization, which works as follows. First we initialize the weights of each (fully connected or convolutional) layer with orthonormal matrices, as proposed in [SMG14]. (This can be achieved by drawing from $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, reshaping to $\boldsymbol{w}$ to a matrix $\mathbf{W}$, and then computing an orthonormal basis using QR or SVD decomposition.) Then, for each layer l, we compute the variance v_l of the activations across a minibatch; we then rescale using $\mathbf{W}_l := \mathbf{W}_l/\sqrt{v_l}$. This scheme can be viewed as an orthonormal initialization combined with batch normalization performed only on the first mini-batch. This is faster than full batch normalization, but can sometimes work just as well.

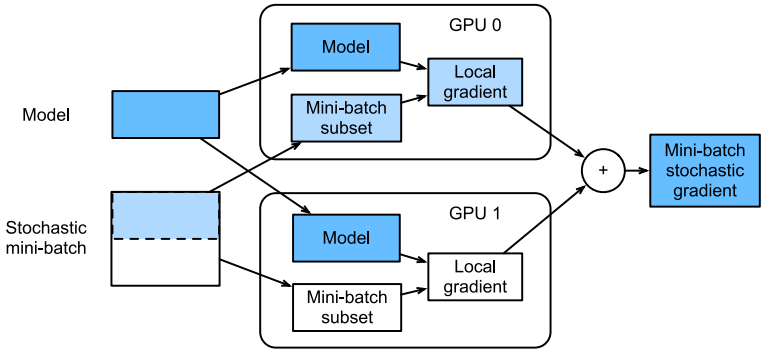


Figure 13.16: Calculation of minibatch stochastic gradient using data parallelism and two GPUs. From Figure 12.5.2 of [Zha+20]. Used with kind permission of Aston Zhang.

13.4.6 Parallel training

It can be quite slow to train large models on large datasets. One way to speed this process up is to use specialized hardware, such as **graphics processing units** (**GPUs**) and **tensor processing units** (**TPUs**), which are very efficient at performing matrix-matrix multiplication. If we have multiple GPUs, we can sometimes further speed things up. There are two main approaches: **model parallelism**, in which we partition the model between machines, and **data parallelism**, in which each machine has its own copy of the model, and applies it to a different set of data.

Model parallelism can be quite complicated, since it requires tight communication between machines to ensure they compute the correct answer. We will not discuss this further. Data parallelism is generally much simpler, since it is **embarassingly parallel**. To use this to speed up training, at each training step t, we do the following: 1) we partition the minibatch across the K machines to get $\mathcal{D}_t^k$; 2) each machine k computes its own gradient, $\boldsymbol{g}_t^k = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_t^k)$; 3) we collect all the local gradients on a central machine (e.g., device 0) and sum them using $\boldsymbol{g}_t = \sum_{k=1}^K \boldsymbol{g}_t^k$; 4) we broadcast the summed gradient back to all devices, so $\tilde{\boldsymbol{g}}_t^k = \boldsymbol{g}_t$; 5) each machine updates its own copy of the parameters using $\boldsymbol{\theta}_t^k := \boldsymbol{\theta}_t^k - \eta_t \tilde{\boldsymbol{g}}_t^k$. See Figure 13.16 for an illustration and multi_gpu_training_jax.ipynb for some sample code.

Note that steps 3 and 4 are usually combined into one atomic step; this is known as an **all-reduce** operation (where we use sum to reduce the set of (gradient) vectors into one). If each machine blocks until receiving the centrally aggregated gradient, $\boldsymbol{g}_t$, the method is known as **synchronous training**. This will give the same results as training with one machine (with a larger batchsize), only faster (assuming we ignore any batch normalization layers). If we let each machine update its parameters using its own local gradient estimate, and not wait for the broadcast to/from the other machines, the method is called **asynchronous training**. This is not guaranteed to work, since the different machines may get out of step, and hence will be updating different versions of the parameters; this approach has therefore been called **hogwild training** [Niu+11]. However, if the updates are sparse, so each machine "touches" a different part of the parameter vector, one can prove that hogwild training behaves like standard synchronous SGD.

13.5 Regularization

In Section 13.4 we discussed computational issues associated with training (large) neural networks. In this section, we discuss statistical issues. In particular, we focus on ways to avoid overfitting. This is crucial, since large neural networks can easily have millions of parameters.

13.5.1 Early stopping

Perhaps the simplest way to prevent overfitting is called **early stopping**, which refers to the heuristic of stopping the training procedure when the error on the validation set starts to increase (see Figure 4.8 for an example). This method works because we are restricting the ability of the optimization algorithm to transfer information from the training examples to the parameters, as explained in [AS19].

13.5.2 Weight decay

A common approach to reduce overfitting is to impose a prior on the parameters, and then use MAP estimation. It is standard to use a Gaussian prior for the weights $\mathcal{N}(\boldsymbol{w}|\mathbf{0}, \alpha^2\mathbf{I})$ and biases, $\mathcal{N}(\boldsymbol{b}|\mathbf{0}, \beta^2\mathbf{I})$. This is equivalent to ℓ_2 regularization of the objective. In the neural networks literature, this is called **weight decay**, since it encourages small weights, and hence simpler models, as in ridge regression (Section 11.3).

13.5.3 Sparse DNNs

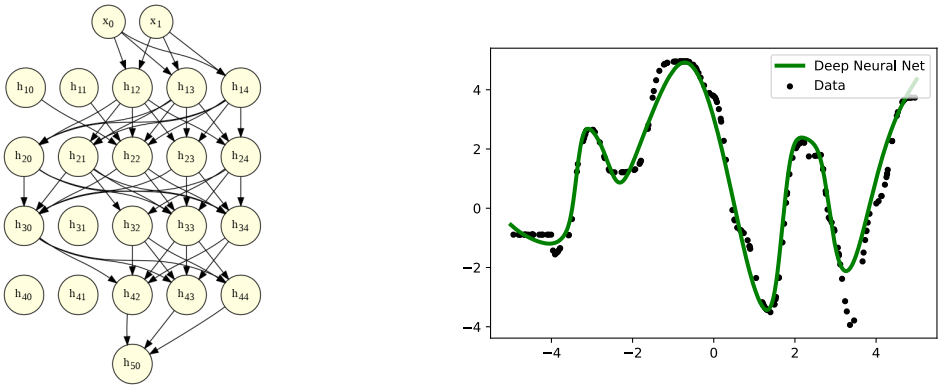
Since there are many weights in a neural network, it is often helpful to encourage sparsity. This allows us to perform **model compression**, which can save memory and time. To do this, we can use ℓ_1 regularization (as in Section 11.4), or ARD (as in Section 11.7.7), or several other methods (see e.g., [Hoe+21; Bha+20] for recent reviews). As a simple example, Figure 13.17 shows a 5 layer MLP which has been fit to some 1d regression data using an ℓ_1 regularizer on the weights. We see that the resulting graph topology is sparse.

Despite the intuitive appeal of sparse topology, in practice these methods are not widely used, since modern GPUs are optimized for *dense* matrix multiplication, and there are few computational benefits to sparse weight matrices. However, if we use methods that encourage *group* sparsity, we can prune out whole layers of the model. This results in *block sparse* weight matrices, which can result in speedups and memory savings (see e.g., [Sca+17; Wen+16; MAV17; LUW17]).

13.5.4 Dropout

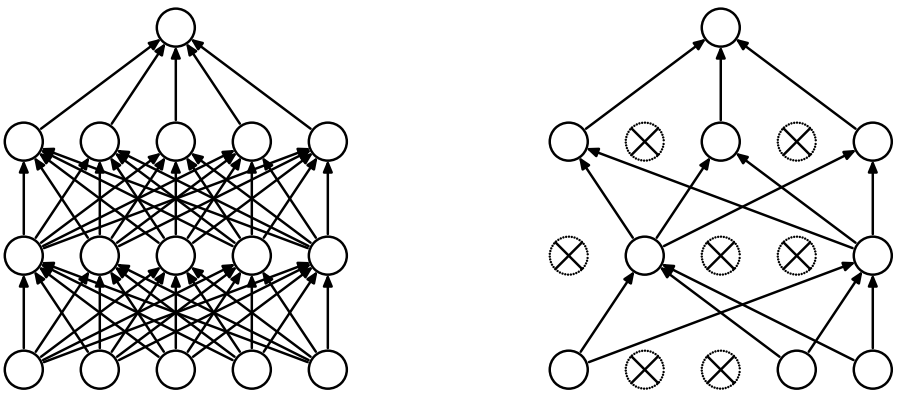
Suppose that we randomly (on a per-example basis) turn off all the outgoing connections from each neuron with probability p, as illustrated in Figure 13.18. This technique is known as **dropout** [Sri+14].

Dropout can dramatically reduce overfitting and is very widely used. Intuitively, the reason dropout works well is that it prevents complex co-adaptation of the hidden units. In other words, each unit must learn to perform well even if some of the other units are missing at random. This prevents the



(a) (b)

Figure 13.17: (a) A deep but sparse neural network. The connections are pruned using ℓ_1 regularization. At each level, nodes numbered 0 are clamped to 1, so their outgoing weights correspond to the offset/bias terms. (b) Predictions made by the model on the training set. Generated by sparse_ mlp.ipynb.



(a) (b)

Figure 13.18: Illustration of dropout. (a) A standard neural net with 2 hidden layers. (b) An example of a thinned net produced by applying dropout with $p_0 = 0.5$. Units that have been dropped out are marked with an x. From Figure 1 of [Sri+14]. Used with kind permission of Geoff Hinton.

units from learning complex, but fragile, dependencies on each other.[7] A more formal explanation, in terms of Gaussian scale mixture priors, can be found in [NHLS19].

We can view dropout as estimating a noisy version of the weights, $\theta_{lji} = w_{lji}\epsilon_{li}$, where $\epsilon_{li} \sim \text{Ber}(1-p)$ is a Bernoulli noise term. (So if we sample $\epsilon_{li} = 0$, then all of the weights going out of unit i in layer $l-1$ into any j in layer l will be set to 0.) At test time, we usually turn the noise off.

7. Geoff Hinton, who invented dropout, said he was inspired by a talk on sexual reproduction, which encourages genes to be individually useful (or at most depend on a small number of other genes), even when combined with random other genes.

To ensure the weights have the same expectation at test time as they did during training (so the input activation to the neurons is the same, on average), at test time we should use $w_{lij} = \theta_{lji}\mathbb{E}[\epsilon_{li}]$. For Bernoulli noise, we have $\mathbb{E}[\epsilon] = 1 - p$, so we should multiply the weights by the keep probability, $1 - p$, before making predictions.

We can, however, use dropout at test time if we wish. The result is an **ensemble** of networks, each with slightly different sparse graph structures. This is called **Monte Carlo dropout** [GG16; KG17], and has the form

$$p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(\boldsymbol{y}|\boldsymbol{x}, \hat{\mathbf{W}}\epsilon^s + \hat{\boldsymbol{b}}) \tag{13.96}$$

where S is the number of samples, and we write $\hat{\mathbf{W}}\epsilon^s$ to indicate that we are multiplying all the estimated weight matrices by a sampled noise vector. This can sometimes provide a good approximation to the Bayesian posterior predictive distribution $p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D})$, especially if the noise rate is optimized [GHK17].

13.5.5 Bayesian neural networks

Modern DNNs are usually trained using a (penalized) maximum likelihood objective to find a single setting of parameters. However, with large models, there are often many more parameters than data points, so there may be multiple possible models which fit the training data equally well, yet which generalize in different ways. It is often useful to capture the induced uncertainty in the posterior predictive distribution. This can be done by *marginalizing out* the parameters by computing

$$p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D}) = \int p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \tag{13.97}$$

The result is known as a **Bayesian neural network** or **BNN**. It can be thought of as an infinite ensemble of differently weight neural networks. By marginalizing out the parameters, we can avoid overfitting [Mac95]. Bayesian marginalization is challenging for large neural networks, but also can lead to significant performance gains [WI20]. For more details on the topic of **Bayesian deep learning**, see the sequel to this book, [Mur23].

13.5.6 Regularization effects of (stochastic) gradient descent *

Some optimization methods (in particular, second-order batch methods) are able to find "needles in haystacks", corresponding to narrow but deep "holes" in the loss landscape, corresponding to parameter settings with very low loss. These are known as **sharp minima**, see Figure 13.19(right). From the point of view of minimizing the empirical loss, the optimizer has done a good job. However, such solutions generally correspond to a model that has overfit the data. It is better to find points that correspond to **flat minima**, as shown in Figure 13.19(left); such solutions are more robust and generalize better. To see why, note that flat minima correspond to regions in parameter space where there is a lot of posterior uncertainty, and hence samples from this region are less able to precisely memorize irrelevant details about the training set [AS17]. SGD often finds such flat minima by virtue of the addition of noise, which prevents it from "entering" narrow regions of the loss landscape (see e.g., [SL18]). This is called **implicit regularization**. It is also possible to explicitly encourage

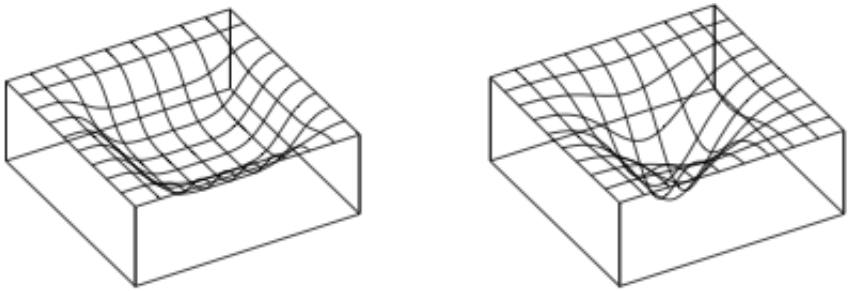
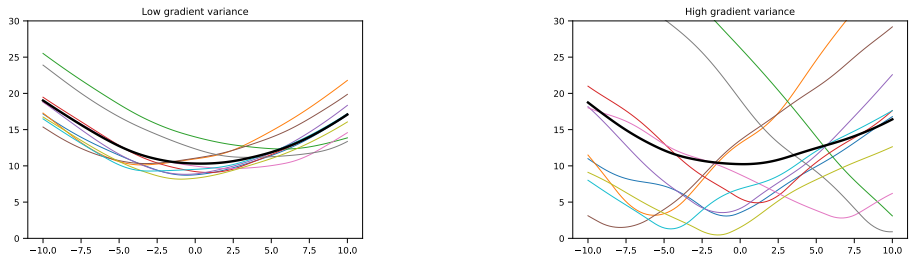


Figure 13.19: Flat vs sharp minima. From Figures 1 and 2 of [HS97a]. Used with kind permission of Jürgen Schmidhuber.



(a) *(b)*

Figure 13.20: Each curve shows how the loss varies across parameter values for a given minibatch. (a) A stable local minimum. (b) An unstable local minimum. Generated by sgd_minima_variance.ipynb. Adapted from https://bit.ly/3wTc1L6.

SGD to find such flat minima, using **entropy SGD** [Cha+17], **sharpness aware minimization** [For+21], **stochastic weight averaging** (SWA) [Izm+18], and other related techniques.

Of course, the loss landscape depends not just on the parameter values, but also on the data. Since we usually cannot afford to do full-batch gradient descent, we will get a set of loss curves, one per minibatch. If each one of these curves corresponds to a wide basin, as shown in Figure 13.20a, we are at a point in parameter space that is robust to perturbations, and will likely generalize well. However, if the overall wide basin is the result of averaging over many different narrow basins, as shown in Figure 13.20b, the resulting estimate will likely generalize less well.

This can be formalized using the analysis in [Smi+21; BD21]. Specifically, they consider continuous time gradient flow which approximates the behavior of (S)GD. In [BD21], they consider full-batch GD, and show that the flow has the form $\dot{\boldsymbol{w}} = -\nabla_{\boldsymbol{w}} \tilde{\mathcal{L}}_{GD}(\boldsymbol{w})$, where

$$\tilde{\mathcal{L}}_{GD}(\boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \frac{\epsilon}{4}||\nabla \mathcal{L}(\boldsymbol{w})||^2 \tag{13.98}$$

where $\mathcal{L}(\boldsymbol{w})$ is the original loss, ϵ is the learning rate, and the second term is an implicit regularization term that penalizes solutions with large gradients (high curvature).

In [Smi+21], they extend this analysis to the SGD case. They show that the flow has the form $\dot{\boldsymbol{w}} = -\nabla_{\boldsymbol{w}} \tilde{\mathcal{L}}_{SGD}(\boldsymbol{w})$, where

$$\tilde{\mathcal{L}}_{SGD}(\boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \frac{\epsilon}{4m} \sum_{k=1}^{m} ||\nabla \mathcal{L}_k(\boldsymbol{w})||^2 \tag{13.99}$$

where m is the number of minibatches, and $\mathcal{L}_k(\boldsymbol{w})$ is the loss on the k'th such minibatch. Comparing this to the full-batch GD loss, we see

$$\tilde{\mathcal{L}}_{SGD}(\boldsymbol{w}) = \tilde{\mathcal{L}}_{GD}(\boldsymbol{w}) + \frac{\epsilon}{4m} \sum_{k=1}^{m} ||\nabla \mathcal{L}_k(\boldsymbol{w}) - \nabla \mathcal{L}(\boldsymbol{w})||^2 \tag{13.100}$$

The second term estimates the variance of the minibatch gradients, which is a measure of stability, and hence of generalization ability.

The above analysis shows that SGD not only has computational advantages (since it is faster than full-batch GD or second-order methods), but also statistical advantages.

13.5.7 Over-parameterized models

Modern neural networks use many more parameters P than training examples N. Such models are said to be **over-parameterized**. As such, they typically achieve zero training loss (at least for classification problems). Despite this, they often generalize better than models where $P \ll N$. In particular, it is observed that models where $P < N$ show the usual U-shaped curve, where test error initially decreases with increasing P, and then increases as the model starts to overfit, as predicted by the bias-variance tradeoff discussed in Section 4.7.6.3. However, when $P > N$, test error starts to decrease again; this is known as the **double descent** phenomenon [HK92; Bel+19; Nak+19]. Intuitively the reason for this is as follows: when $P = N$, the model may need to represent a very non-smooth function to interpolate the data (see Figure 1.7c), but when $P \gg N$, it can interpolate the data (thus getting zero training error) using a smooth function [BS21]. See [DMB21] for details.

13.6 Other kinds of feedforward networks *

13.6.1 Radial basis function networks

Consider a 1 layer neural net where the hidden layer is given by the feature vector

$$\boldsymbol{\phi}(\boldsymbol{x}) = [\mathcal{K}(\boldsymbol{x}, \boldsymbol{\mu}_1), \ldots, \mathcal{K}(\boldsymbol{x}, \boldsymbol{\mu}_K)] \tag{13.101}$$

where $\boldsymbol{\mu}_k \in \mathcal{X}$ are a set of K **centroids** or **exemplars**, and $\mathcal{K}(\boldsymbol{x}, \boldsymbol{\mu}) \geq 0$ is a **kernel function**. We describe kernel functions in detail in Section 17.1. Here we just give an example, namely the **Gaussian kernel**

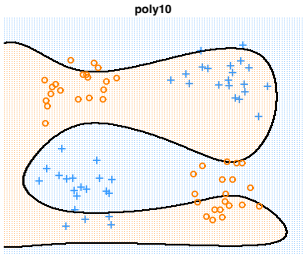
$$\mathcal{K}_{\text{gauss}}(\boldsymbol{x}, \boldsymbol{c}) \triangleq \exp\left(-\frac{1}{2\sigma^2} ||\boldsymbol{c} - \boldsymbol{x}||_2^2\right) \tag{13.102}$$

The parameter σ is known as the **bandwidth** of the kernel. Note that this kernel is shift invariant, meaning it is only a function of the distance $r = ||\boldsymbol{x} - \boldsymbol{c}||_2$, so we can equivalently write this as

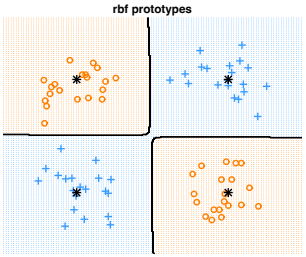
$$\mathcal{K}_{\text{gauss}}(r) \triangleq \exp\left(-\frac{1}{2\sigma^2} r^2\right) \tag{13.103}$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

(a)



(b)



(c)

Figure 13.21: (a) xor truth table. (b) Fitting a linear logistic regression classifier using degree 10 polynomial expansion. (c) Same model, but using an RBF kernel with centroids specified by the 4 black crosses. Generated by logregXorDemo.ipynb.

This is therefore called a **radial basis function kernel** or **RBF kernel**.

A 1 layer neural net in which we use Equation (13.101) as the hidden layer, with RBF kernels, is called an **RBF network** [BL88]. This has the form

$$p(y|\boldsymbol{x};\boldsymbol{\theta}) = p(y|\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x})) \tag{13.104}$$

where $\boldsymbol{\theta} = (\boldsymbol{\mu}, \boldsymbol{w})$. If the centroids $\boldsymbol{\mu}$ are fixed, we can solve for the optimal weights $\boldsymbol{w}$ using (regularized) least squares, as discussed in Chapter 11. If the centroids are unknown, we can estimate them by using an unsupervised clustering method, such as K-means (Section 21.3). Alternatively, we can associate one centroid per data point in the training set, to get $\boldsymbol{\mu}_n = \boldsymbol{x}_n$, where now $K = N$. This is an example of a **non-parametric model**, since the number of parameters grows (in this case linearly) with the amount of data, and is not independent of N. If $K = N$, the model can perfectly interpolate the data, and hence may overfit. However, by ensuring that the output weight vector $\boldsymbol{w}$ is sparse, the model will only use a finite subset of the input examples; this is called a **sparse kernel machine**, and will be discussed in more detail in Section 17.4.1 and Section 17.3. Another way to avoid overfitting is to adopt a Bayesian approach, by integrating out the weights $\boldsymbol{w}$; this gives rise to a model called a **Gaussian process**, which will be discussed in more detail in Section 17.2.

13.6.1.1 RBF network for regression

We can use RBF networks for regression by defining $p(y|\boldsymbol{x},\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{w}^T\boldsymbol{\phi}(\boldsymbol{x}), \sigma^2)$. For example, Figure 13.22 shows a 1d data set fit with $K = 10$ uniformly spaced RBF prototypes, but with the bandwidth ranging from small to large. Small values lead to very wiggly functions, since the predicted function value will only be non-zero for points $\boldsymbol{x}$ that are close to one of the prototypes $\boldsymbol{\mu}_k$. If the bandwidth is very large, the design matrix reduces to a constant matrix of 1's, since each point is equally close to every prototype; hence the corresponding function is just a straight line.

13.6.1.2 RBF network for classification

We can use RBF networks for binary classification by defining $p(y|\boldsymbol{x},\boldsymbol{\theta}) = \text{Ber}(\sigma(\boldsymbol{w}^T\boldsymbol{\phi}(\boldsymbol{x})))$. As an example, consider the data coming from the exclusive or function. This is a binary-valued function

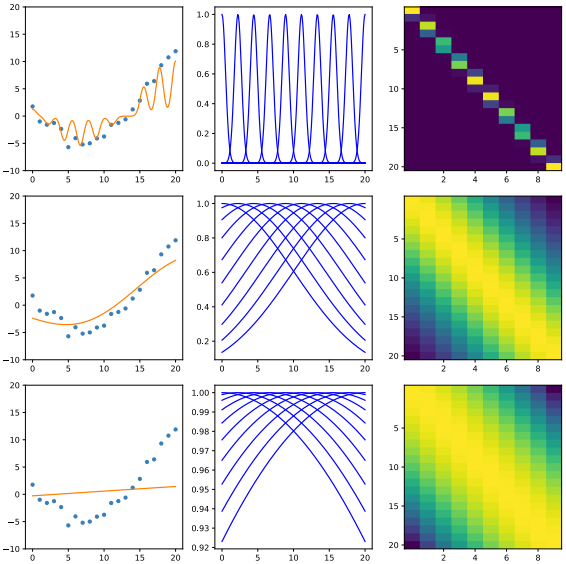


Figure 13.22: Linear regression using 10 equally spaced RBF basis functions in 1d. Left column: fitted function. Middle column: basis functions evaluated on a grid. Right column: design matrix. Top to bottom we show different bandwidths for the kernel function: $\sigma = 0.5, 10, 50$. Generated by linregRbfDemo.ipynb.

of two binary inputs. Its truth table is shown in Figure 13.21(a). In Figure 13.21(b), we have shown some data labeled by the xor function, but we have **jittered** the points to make the picture clearer.[8] We see we cannot separate the data even using a degree 10 polynomial. However, using an RBF kernel and just 4 prototypes easily solves the problem as shown in Figure 13.21(c).

13.6.2 Mixtures of experts

When considering regression problems, it is common to assume a unimodal output distribution, such as a Gaussian or Student distribution, where the mean and variance is some function of the input, i.e.,

$$p(\boldsymbol{y}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}|f_\mu(\boldsymbol{x}), \text{diag}(\sigma_+(f_\sigma(\boldsymbol{x})))) \tag{13.105}$$

where the f functions may be MLPs (possibly with some shared hidden units, as in Figure 13.5). However, this will not work well for **one-to-many functions**, in which each input can have multiple possible outputs.

Figure 13.23a gives a simple example of such a function. We see that in the middle of the plot there are certain x values for which there are two equally probable y values. There are many real world problems of this form, e.g., 3d pose prediction of a person from a single image [Bo+08], colorization

8. Jittering is a common visualization trick in statistics, wherein points in a plot/display that would otherwise land on top of each other are dispersed with uniform additive noise.

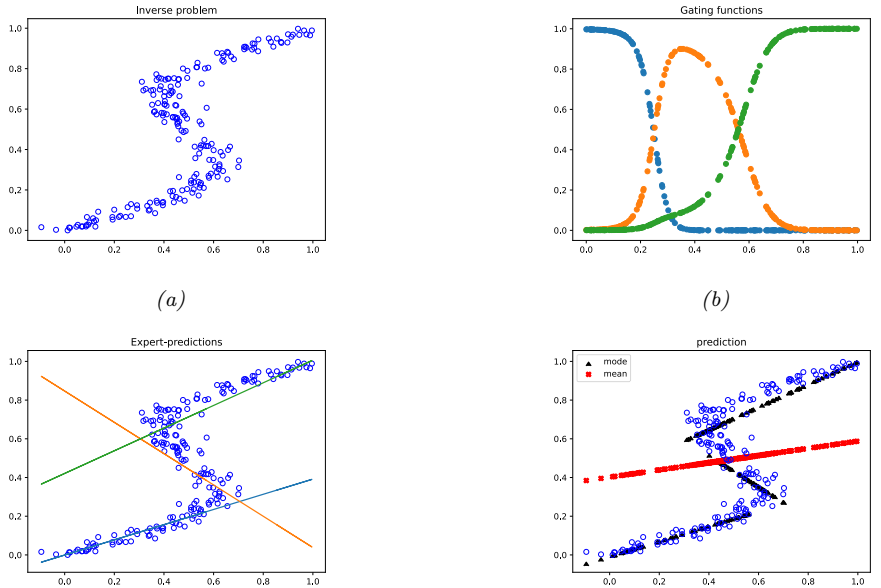


Figure 13.23: (a) Some data from a one-to-many function. Horizontal axis is the input x, vertical axis is the target $y = f(x)$. (b) The responsibilities of each expert for the input domain. (c) Prediction of each expert (colored lines) superimposed on the training data. (d) Overall prediction. Mean is red cross, mode is black square. Adapted from Figures 5.20 and 5.21 of [Bis06]. Generated by mixexpDemoOneToMany.ipynb.

of a black and white image [Gua+17], predicting future frames of a video sequence [VT17], etc. Any model which is trained to maximize likelihood using a unimodal output density — even if the model is a flexible nonlinear model, such as neural network — will work poorly on one-to-many functions such as these, since it will just produce a blurry average output.

To prevent this problem of regression to the mean, we can use a **conditional mixture model**. That is, we assume the output is a weighted mixture of K different outputs, corresponding to different modes of the output distribution for each input $\boldsymbol{x}$. In the Gaussian case, this becomes

$$p(\boldsymbol{y}|\boldsymbol{x}) = \sum_{k=1}^{K} p(\boldsymbol{y}|\boldsymbol{x}, z=k)p(z=k|\boldsymbol{x}) \tag{13.106}$$

$$p(\boldsymbol{y}|\boldsymbol{x}, z=k) = \mathcal{N}(\boldsymbol{y}|f_{\mu,k}(\boldsymbol{x}), \text{diag}(f_{\sigma,k}(\boldsymbol{x}))) \tag{13.107}$$

$$p(z=k|\boldsymbol{x}) = \text{Cat}(z|\text{softmax}(f_z(\boldsymbol{x}))) \tag{13.108}$$

Here $f_{\mu,k}$ predicts the mean of the k'th Gaussian, $f_{\sigma,k}$ predicts its variance terms, and f_z predicts which mixture component to use. This model is called a **mixture of experts** (**MoE**) [Jac+91; JJ94; YWG12; ME14]. The idea is that the k'th submodel $p(\boldsymbol{y}|\boldsymbol{x}, z=k)$ is considered to be an "expert" in a certain region of input space. The function $p(z=k|\boldsymbol{x})$ is called a **gating function**, and

decides which expert to use, depending on the input values. By picking the most likely expert for a given input $\boldsymbol{x}$, we can "activate" just a subset of the model. This is an example of **conditional computation**, since we decide what expert to run based on the results of earlier computations from the gating network [Sha+17].

We can train this model using SGD, or using the EM algorithm (see Section 8.7.3 for details on the latter method).

13.6.2.1 Mixture of linear experts

In this section, we consider a simple example in which we use linear regression experts and a linear classification gating function, i.e., the model has the form:

$$p(y|\boldsymbol{x}, z = k, \boldsymbol{\theta}) = \mathcal{N}(y|\boldsymbol{w}_k^\mathsf{T}\boldsymbol{x}, \sigma_k^2) \tag{13.109}$$

$$p(z = k|\boldsymbol{x}, \boldsymbol{\theta}) = \text{Cat}(z|\text{softmax}_k(\mathbf{V}\boldsymbol{x})) \tag{13.110}$$

where softmax_k is the k'th output from the softmax function. The individual weighting term $p(z = k|\boldsymbol{x})$ is called the **responsibility** for expert k for input $\boldsymbol{x}$. In Figure 13.23b, we see how the gating networks softly partitions the input space amongst the $K = 3$ experts.

Each expert $p(y|\boldsymbol{x}, z = k)$ corresponds to a linear regression model with different parameters. These are shown in Figure 13.23c.

If we take a weighted combination of the experts as our output, we get the red curve in Figure 13.23d, which is clearly is a bad predictor. If instead we only predict using the most active expert (i.e., the one with the highest responsibility), we get the discontinuous black curve, which is a much better predictor.

13.6.2.2 Mixture density networks

The gating function and experts can be any kind of conditional probabilistic model, not just a linear model. If we make them both DNNs, then resulting model is called a **mixture density network** (**MDN**) [Bis94; ZS14] or a **deep mixture of experts** [CGG17]. See Figure 13.24 for a sketch of the model.

13.6.2.3 Hierarchical MOEs

If each expert is itself an MoE model, the resulting model is called a **hierarchical mixture of experts** [JJ94]. See Figure 13.25 for an illustration of such a model with a two level hierarchy.

An HME with L levels can be thought of as a "soft" decision tree of depth L, where each example is passed through every branch of the tree, and the final prediction is a weighted average. (We discuss decision trees in Section 18.1.)

13.7 Exercises

Exercise 13.1 [Backpropagation for a MLP]

(Based on an exercise by Kevin Clark.)

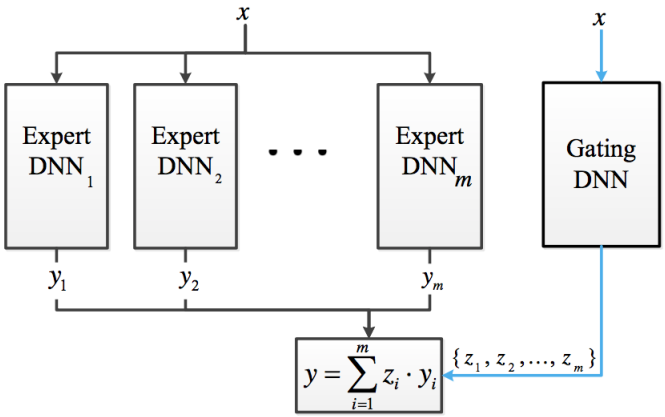


Figure 13.24: Deep MOE with m experts, represented as a neural network. From Figure 1 of [CGG17]. Used with kind permission of Jacob Goldberger.

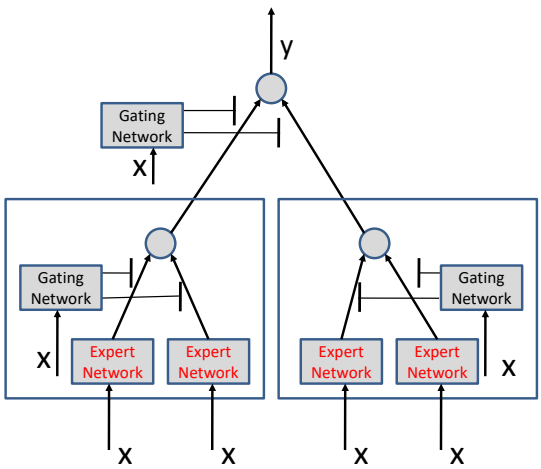


Figure 13.25: A 2-level hierarchical mixture of experts as a neural network. The top gating network chooses between the left and right expert, shown by the large boxes; the left and right experts themselves choose between their left and right sub-experts.

Consider the following classification MLP with one hidden layer:

$$\boldsymbol{x} = \text{input} \in \mathbb{R}^D \tag{13.111}$$

$$\boldsymbol{z} = \mathbf{W}\boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^K \tag{13.112}$$

$$\boldsymbol{h} = \text{ReLU}(\boldsymbol{z}) \in \mathbb{R}^K \tag{13.113}$$

$$\boldsymbol{a} = \mathbf{U}\boldsymbol{h} + \boldsymbol{b}_2 \in \mathbb{R}^C \tag{13.114}$$

$$\mathcal{L} = \text{CrossEntropy}(\boldsymbol{y}, \text{softmax}(\boldsymbol{a})) \in \mathbb{R} \tag{13.115}$$

where $\boldsymbol{x} \in \mathbb{R}^D$, $\boldsymbol{b}_1 \in \mathbb{R}^K$, $\mathbf{W} \in \mathbb{R}^{K \times D}$, $\boldsymbol{b}_2 \in \mathbb{R}^C$, $\mathbf{U} \in \mathbb{R}^{C \times K}$, where D is the size of the input, K is the number of hidden units, and C is the number of classes. Show that the gradients for the parameters and input are as follows:

$$\nabla_{\mathbf{V}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{V}}\right]_{1,:} = \boldsymbol{\delta}_1 \boldsymbol{h}^\mathsf{T} \in \mathbb{R}^{C \times K} \tag{13.116}$$

$$\nabla_{\boldsymbol{b}_2} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_2}\right)^\mathsf{T} = \boldsymbol{\delta}_1 \in \mathbb{R}^C \tag{13.117}$$

$$\nabla_{\mathbf{W}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{W}}\right]_{1,:} = \boldsymbol{\delta}_2 \boldsymbol{x}^\mathsf{T} \in \mathbb{R}^{K \times D} \tag{13.118}$$

$$\nabla_{\boldsymbol{b}_1} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_1}\right)^\mathsf{T} = \boldsymbol{\delta}_2 \in \mathbb{R}^K \tag{13.119}$$

$$\nabla_{\boldsymbol{x}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}}\right)^\mathsf{T} = \mathbf{W}^\mathsf{T}\, \boldsymbol{\delta}_2 \in \mathbb{R}^D \tag{13.120}$$

where the gradients of the loss wrt the two layers (logit and hidden) are given by the following:

$$\boldsymbol{\delta}_1 = \nabla_{\boldsymbol{a}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{a}}\right)^\mathsf{T} = (\boldsymbol{p} - \boldsymbol{y}) \in \mathbb{R}^C \tag{13.121}$$

$$\boldsymbol{\delta}_2 = \nabla_{\boldsymbol{z}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}\right)^\mathsf{T} = (\mathbf{V}^\mathsf{T}\, \boldsymbol{\delta}_1) \odot H(\boldsymbol{z}) \in \mathbb{R}^K \tag{13.122}$$

where H is the Heaviside function. Note that, in our notation, the gradient (which has the same shape as the variable with respect to which we differentiate) is equal to the Jacobian's transpose when the variable is a vector and to the first slice of the Jacobian when the variable is a matrix.